

An Explainable Casting Defect Detection Framework Using CNN, MobileNetV2

Lalith Bhushan Mandhadapu

*Department of DataScience, University of Europe for Applied Sciences, Innovation
Hub, Potsdam, 14469, Brandenburg, Germany*

Abstract

Casting defect detection through image classification plays an important role in computer vision. It is used inspection process in various industries for quality of the product. Accurately identifying defects in casting process through images can reduce time taken for the inspection through machine or by experienced work men and increase efficiency in defect detection. Many deep learning techniques have successfully achieved high performance in image classification tasks. Existing studies focus mainly on model productivity, with limited interpretability of model decisions. Moreover, comparisons between convolutional neural network models and transfer learning models are relatively limited, motivating the development of an explainable classification framework. For this project, the dataset was obtained from the Kaggle, which contains two classes. Two learning models, namely Convolutional Neural Networks (CNNs), MobileNetV2, were implemented and evaluated using accuracy, loss, F1-score, and confusion matrix. Experimental results demonstrated that transfer learning models performed better than the CNN model in classifying vehicle images. MobileNetV2 achieved the highest validation accuracy, approximately 97.7% where as custom cnn has achieved an accuracy of 95%. GRAD-CAM Explainable Artificial Intelligence (XAI) analysis provided insights into the model's decision-making by identifying image regions that contributed to the classification outcomes. The proposed framework demonstrates that combining Explainable AI with transfer learning models provides highly accurate results for casting defect detection, making it more useful for inspection processes in a manufacturing industry.

Keywords: GRAD-CAM, Convolutional Neural Network, MobileNetV2

1. Graphical Abstract

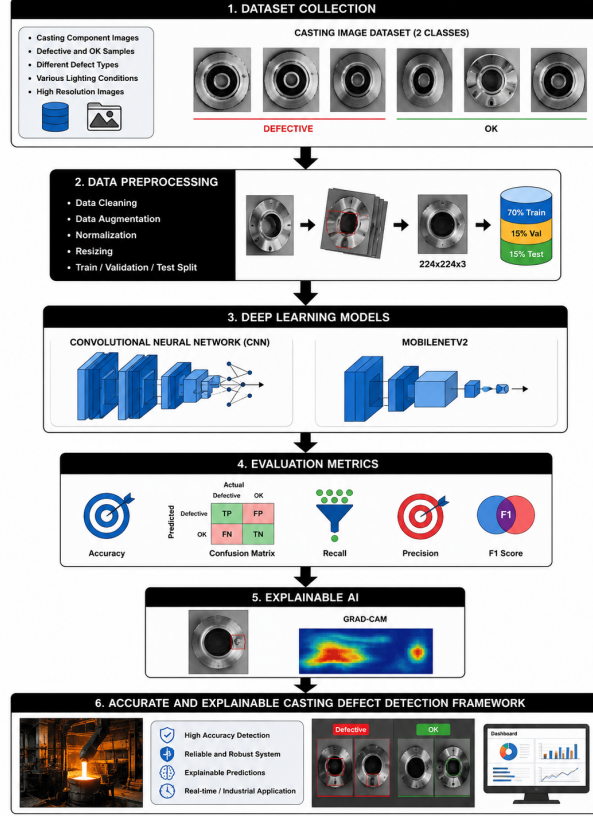


Figure 1: Graphical abstract representation of the proposed casting defect detection framework.

2. Highlights

- Developed a machine learning model to detect defects in casting of machine parts.
- Trained MobileNetV2, CNN models using the Kaggle casting defect detection dataset.
- Applied transfer learning to speed up the training and improve the prediction accuracy.
- Used GRAD-CAM to achieve interpretable prediction results for the image classification task.

- Chosen MobileNetV2 as the optimal model for this problem.

3. Introduction

3.1. Application Background and Practical Importance

Automating Casting Defect Detection Using CNN And Transfer Learning Models Detecting and analyzing imperfections that appear during the casting process, a critical manufacturing stage for producing mechanical parts in industries such as aerospace, automotive, heavy machinery and rail transportation. Product defects found in cast components impact quality, safety, performance and service life, for instance rough surface textures, shrinking defects, cracked structures, pores or voids. Due to non-uniform mold temperatures, insufficient pre-processing of mold components, inclusions in molten metal and variability during metal casting process the surface quality can differ.

Thus, a robust system capable of efficiently inspecting cast products, eliminating issues due to human limitations or variability is needed.

Due to current trends in Industry 4.0 which push towards intelligent production lines, systems capable of analyzing images automatically in real-time, without relying on visual inspection procedures, are of high demand. Artificial Intelligence technologies, and deep learning in particular, have enabled rapid advances in automated visual inspection solutions. The most promising application of this area, Convolutional Neural Networks (CNNs), have proven effective in the classification of images and identification of objects by automatically learning high-level visual features directly from the raw data, such as color, edges and textures. These features enable CNN models to discern subtle nuances of surface patterns, texture variations and structural features, thus differentiating acceptable products from defective ones.

Therefore, CNN-based image analysis systems have achieved state-of-the-art performance across many industrial applications.

Custom trained CNN models offer a high classification performance. However, in the absence of sufficient datasets to train custom CNNs from scratch, transfer learning models like MobileNetV2 and EfficientNetB0 have demonstrated excellent classification accuracy with small datasets by adapting pre-trained networks, developed on a large general image dataset. The efficiency of MobileNetV2 makes it suitable for real-time industrial inspection, while EfficientNetB0 provide an adequate trade-off between efficiency and classification power.

We compare these two models with a custom CNN on detecting and classifying imperfections in cast components. In this paper, an explainable casting defect detection framework combining a custom CNN with two well-known transfer learning models (MobileNetV2 and EfficientNetB0) is proposed. This framework integrates an image pre-processing module, a data augmentation module and a model validation scheme using metrics such as accuracy, precision, recall, F1-score, and confusion matrix, while Grad-CAM (Gradient-weighted Class Activation Mapping) is incorporated to make the model interpretable by visualizing the region of image that most significantly contributed to the prediction.

This will help improve the transparency of our system for quality control and will contribute to intelligent manufacturing systems development.

3.2. Machine Learning and Computer Vision Context

- The advent of machine learning has facilitated a revolution in analysing images across an assortment of fields, such as travel frameworks, medical imaging, agricultural applications, production, and security. For tasks of image categorisation, the main target is to automatically correlate an image with one of various assigned categories as predicated upon its visual characteristics. Typical ML techniques have commonly utilised feature construction, which necessitated human expertise, and have seldom resulted in satisfactory generalisation for images under varying circumstances. Because of the increasing sizes and complexity of datasets now available for image-based training, the learning methods of automating the identification of features have become incredibly important.
- Computer vision describes the development of systems capable of processing and interpreting visible information obtained from photos and video recordings. Computer vision tasks involving the images of casting defects can be identified with a system tasked with accurately identifying the detects in the product according to physical visual details that include, but are not limited to, their shapes, colours, appearances and texture. Diverse points of view, image quality, varying lighting, and highly complex backgrounds add complexity and demand the employment of powerful feature extraction approaches.
- CNNs have proved themselves as one of the most successful deep learning structures when used with image analysis and classification.

- Unnecessary traditional approaches that rely on features manually engineered from their surroundings utilise, rather, learning methods wherein the CNN acquires appropriate visible feature patterns from raw image data throughout the training process. Through repeated convolution operations across numerous layers, the CNN can discern the lowest-level qualities, such as shape and texture; then middle-stage features, such as irregularity and appearances; and finally, the highest-level features that reflect complete object models. The process of acquiring knowledge from images through these series of layers allows CNNs the capacity to master substantial visible info effectively. A substantial advantage associated with CNNs is their capacity to learn the area dependencies between images. The convolution operations help in making patterns discoverable despite position in image information, regardless of the precise area.
- Pooling techniques then enhance resilience to positioning and scale variances. This implies that CNNs have the ability to discover and recognise crucial elements that would generally improve classification accuracy.
- In this study, it can be seen how to benefit from pre-existing and well-established CNN-based architectures. The concept is to learn a robust feature representation on a large collection of photographs to train a generic model for defect identification, which is then specialised to distinguish the several vehicle types described earlier. This idea, the so-called 'learning by transfer' (which allows a transfer learning paradigm), means that the system can reuse an architecture already optimised on a set of general images and is expected to obtain the expected good performance. So, using pre-trained CNN architectures, you will be able to benefit from knowledge previously learned by big databases and adapt it to your problem. Hence, the use of CNN-based deep learning models has enabled this task.

3.3. *Transfer Learning for the Selected Problem*

- **Transfer Learning for the Selected Problem** The training of a deep convolutional neural network requires huge numbers of labelled images and significant computation from random initialisation. Deep networks learn visual representation and features from sufficient training images.

However, many vehicle image classification tasks require thousands of labelled high-resolution images that would be impractical or extremely costly to acquire.

- When limited training data is available, the overfitting problem can arise as networks trained from scratch may fail to generalise to unobserved images. The difficulty of this task is further compounded by issues like variations in vehicle appearance, including size, orientation and angle; differing environmental conditions, including illumination, background environment and scale; and varying image quality. Therefore, an alternative to building deep networks from scratch can be transfer learning. Transfer learning is a way to take knowledge learned from large and well-annotated image databases and reuse and adapt this knowledge for different image classification tasks. Knowledge learned from very large databases of general-purpose pre-learning colour images helps in pre-learning various edge, texture, colour, contrast features; contours; spatial locations of object parts; and the overall arrangement of the objects, and then the transfer learner can reuse low-level and high-level feature representation learning from the pre-learned models for the new problems.
- As the model uses some or all of the already learned features, the training can be performed with fewer training examples. Furthermore, there will be less and faster convergence time. Thus, transfer learning becomes popular for image classification with limited labelled datasets. Transfer learning can be achieved in two different ways: feature extraction and fine-tuning.
- The final classifier layer of the pre-trained network would be retrained with its parameters, and then this whole retrained system would use previously obtained weights (non-learnable or with a small learning rate) in deeper layers as a feature extractor. The pre-trained network is either used as a feature extractor or its weights are initialised from it. During fine-tuning, some of the layers of the pre-trained network are trained with some data.
- The MobileNetV2 were chosen as transfer learning models since they are computationally affordable and give competitive results on a variety of image classification problems.

3.4. *Explainable AI and Trustworthiness*

Although high classification accuracy is a necessary indicator, a well-performed prediction cannot provide any information on how the result was obtained. High performance (accuracy) on the test set can be achieved even with irrelevant image features, so it is impossible to detect. Deep learning models can be called "black-box" systems because human intelligibility is hindered. An image classification model might learn that vehicles of certain types, for instance, always appear in certain backgrounds or under specific illumination and, in turn, focus on image regions that are indicative of such context or environment instead of focusing on features of the car itself. This type of phenomenon is known as shortcut learning, whereby the model ends up leveraging correlations in the dataset that are not reflective of the true object and is hence not a good indicator of predictive accuracy that can be attributed to knowledge about car classes itself. For this reason, when interpreting performance metrics alone, as may often be observed for models designed in the image classification pipeline.

The Explainable AI (XAI) methodology has thus been designed and developed as the countermeasure. It helps us to discover what parts of an image influence a model to make a certain decision; these are the so-called "saliency maps", which would, in our case of vehicles, inform us what parts (e.g., wheel, body structure, windows or other distinguishing parts) a model deems relevant to conclude a certain car type is identified. By doing so, if the attention of the model is on the background images, for example, instead of the car, we would consider such a result as evidence of overfitting due to the dataset bias or simply of poor generalising capability. XAI methodology is very important, particularly in applications of machine learning in which the system has been developed for making decisions in real-world settings. It offers insights that go beyond simple metrics of predictive performance like accuracy, precision, recall and F1-score.

Grad-CAM and SHAP are two of the most prevalent approaches to explainability. Grad-CAM produces a visualisation that highlights the regions of an image that were most influential in determining the prediction made. Using Grad-CAM, we can verify this via visualisation. Whereas Grad-CAM focuses on locating the features that have generated the prediction. By implementing this explanation approach, we could gain more trust in the robustness of the prediction process.

We have seen how the accuracy is not enough for judging a model. Nevertheless, without explainable outcomes of this classifier, there could be a

series of very practical issues arising if we decide to apply it in a real-world environment. For instance, any manufacturing application that banks on automated casting defect detection, like quality inspection , production line monitoring , or even intelligent manufacturing systems , really depends on getting every single casting product right, including the ones with tiny, subtle or hard to spot defects. With dependable automatic defect detection, you can help make sure the product quality stays solid , cut down the amount of manual checking that people have to do, reduce overall production costs, and in the end improve manufacturing efficiency in general.

3.5. Research Gap

Even though deep learning has been really helpful for automated casting defect detection, there are still some issues that stay, even with the recent progress. A lot of existing approaches depend on manual or handcrafted feature extraction, or they use deep neural networks that are quite complex and they also need a lot of computing power and huge labeled datasets. In practice these methods can have trouble spotting tiny surface defects, especially when lighting conditions shift, when image noise appears, and when the casting orientation changes . On top of that, many papers seem to concentrate mainly on pushing classification accuracy higher , without really caring about computational efficiency. So, those pipelines can be less suitable for real-time industrial inspection systems, where timing matters.

Also, there is not enough side-by-side comparison work that looks at lightweight transfer learning models versus custom convolutional neural networks (CNNs) specifically for binary casting defect classification. It’s important to see the trade-off among accuracy, computational burden and inference speed, because deployment decisions in manufacturing aren’t only about “best accuracy”, they are about what runs fast enough, and reliably.

This research tries to fill those gaps by building and comparing a custom CNN together with a MobileNetV2 transfer learning model for automated casting defect detection. The overall goal is to keep strong classification accuracy, but also maintain computational efficiency, so the method can support intelligent, near real-time quality inspection across industrial production lines.

3.6. Problem Statement

In today’s manufacturing industries, the quality of the casting products is essential to the reliability, safety and performance of the final product.

The traditional casting defect inspection mainly relies on manual visual inspection, and the inspection process is time-consuming, labor-intensive, and prone to human error, especially for the detection of small and subtle surface defects. Such restrictions may result in variability in quality control, higher manufacturing expenses, and production delays.

While some automated inspection methods have been developed, most of them are based on hand-crafted image features or are based on deep learning algorithms which are computationally expensive and not suitable for real-time industrial use. High detection accuracy with high computational efficiency is still a great challenge.

So, there is a need for an intelligent, automatic casting defect detection system which can detect the casting defect with high accuracy and works efficiently in an industrial environment. To solve this issue, this study introduces and evaluates a Custom Convolutional Neural Network (CNN) and a transfer learning method using MobileNetV2 for casting defect classification to find the best solution for accurate and efficient defect detection.

3.7. Aim and Objectives

This study aims to design, develop, and evaluate an explainable CNN-based framework for the task of accurate casting defect detection. To fulfil the aim, the study has multiple objectives as follows:

1. Analysing the selected casting defect detection images dataset and pre-processing it for enhancing the quality of data and the quality of the model.
2. Training a custom CNN model that can provide a baseline to be compared with other methods.
3. Training and validating transfer learning-based models, including MobileNetV2.
4. Estimating the performance of the aforementioned models using different evaluation metrics like accuracy, precision, recall, F1-score, and confusion matrices.
5. Utilising explainable techniques, such as Grad-CAM to explain predictions made by the models and examine the transparency of the framework.

3.8. *Research Questions*

- How well a Custom Convolutional Neural Network (CNN) can classify casting products as defective or not defective from industrial image datasets?
- To test the effectiveness of the MobileNetV2 transfer learning model for casting defect detection using classification accuracy, precision, recall and F1-score analysis to compare it with the Custom CNN.
- What are the effects of image preprocessing and data augmentation methods on deep learning models for defect classification in casting?
- Compared with the Custom CNN model, what are the differences in computational efficiency, training time, and speed between the model and the MobileNetV2 model for the automated industrial quality inspection?
- What is the optimal balance between accuracy and computational efficiency for deep learning modelling in real-time casting defect detection in manufacturing?

3.9. *Contributions and Novelty*

The paper introduces a transparent automated system for detecting casting defects combining the concepts of deep learning, transfer learning and model interpretability. This study systematically compares the accuracy of the classification model of a Custom Convolutional Neural Network (CNN) with the MobileNetV2 transfer learning model, but also analyzes their computational efficiency, reliability, and suitability for industrial quality inspection. A pipeline capable of being repeated was created including image pre-processing, data augmentation, model training, validation, and model performance evaluation.

To evaluate the performance of both models, a comparative analysis was carried out using a number of evaluation metrics with the performance of the Custom CNN and MobileNetV2 compared against defective and non-defective casting products. To make the deep learning models more transparent, Grad-CAM was used to see the image regions which most affect the classification, and understand the process through which the model makes its prediction.

Both models were assessed for accuracy, precision, recall, f1-score, confusion matrix, ROC-AUC and detailed error analysis to identify the strengths and weaknesses of the models on a class-by-class basis. Moreover, the trade-off between classification performance and computational complexity was investigated to find the most appropriate model for the real-time application of automated industrial casting quality inspection systems.

3.10. Report Organization

The rest of this paper is organized as follows. The Literature Review describes the previous work on automated casting defect detection, convolutional neural networks (CNNs), transfer learning and explainable artificial intelligence (XAI) for industrial quality inspection. The ****Methodology**** section provides a description of the casting defect dataset, image preprocessing, data augmentation technique, model development, training procedure and methodology of evaluation. The Results section provides performance of the Custom CNN and MobileNetV2 models, comparative analysis, classification performance, and interpretability of the models by using the Grad-CAM visualizations. The Discussions will address the applicability of the proposed solution, the advantages and disadvantages of the models, and the feasibility of their real-time application in the field of industrial quality control. Last but not least, the study’s main findings are summarized in the Conclusion and directions for future research in automated casting defect detection are provided.

4. Literature Review

In the era of Industry 4.0, the use of Artificial Intelligence (AI) and Deep Learning approaches in automating Quality Inspection in manufacturing industries is growing at a rapid pace. Artificial Intelligence (AI) and Deep Learning techniques are rapidly being used in manufacturing industries to automate Quality Inspection, and with the emergence of Industry 4.0, the use of these technologies is increasing exponentially. Casting is one of the most significant manufacturing processes that is used to create the complex metal cast components which are used in the automobile, aerospace, construction, heavy engineering, and other sectors. These defects such as porosity, blow holes, shrinkage cavities, cracks and inclusions, are all very important to the mechanical properties and reliability of casting products. Traditional visual

detection techniques are labor-intensive and time-consuming with human error and the need for more intelligent defect detection systems is growing. In Industry, the efficiency and accuracy of quality inspection systems have been greatly enhanced by recent advancements in Convolutional Neural Networks (CNNs) and transfer learning [1].

Habibpour et al. [2] proposed a deep learning approach for defect detection in casting that uses transfer learning to calculate the uncertainty. Four pre-trained CNN architectures (VGG16, ResNet50, DenseNet121 and InceptionResNetV2) were used for feature extraction and machine learning classifiers (Support Vector Machines (SVM) and Multi-Layer Perceptrons (MLP)) were used. The proposed framework estimated predictive uncertainty, thus increasing the confidence of the automatic inspection systems, compared to conventional methods based on CNN that only provide class predictions. Experimental results showed that transfer learning gave a significant improvement in feature representation on small datasets and uncertainty estimation gave a better trustworthiness in quality inspection. The multi-stage architecture, however, is much more complex in terms of calculation and not very suitable in real-time industrial environments.

In this context, Tanyıldız and Batur Şahin [1] studied the use of transfer learning in automated detection of casting defect in Industry 4.0. The goal of this study was to help mitigate the issues associated with manual inspection by feature extraction using MobileNetV2, ResNET50 and InceptionV3 models and then classification of the extracted model using a cubic Support Vector Machine. The model they proposed not only yielded good classification accuracy but also showed that transfer learning could be a promising method for automating the quality inspection process with a reduction in production errors and inspection time. However, the study was mainly focused on maximizing the classification accuracy with hybrid architectures, and did not explore the balancing act between computational efficiency and simplicity of the model needed for its practical use in production environments.

More recent CNN-based studies have also shown that deep learning can be used to effectively classify casting products that are defective or not defective without having to handcraft features to be used for classification. CNNs are trained in a hierarchical way to automatically extract the visual features of the images, which are more robust than traditional machine vision systems under the presence of illumination variations, texture and defect appearance variations. Although these methods achieved good classifica-

tion performance, most studies only evaluated a single DL architecture and provided limited comparisons between the light-weight transfer learning architectures and custom-designed CNNs. Therefore, the development of the optimal architecture for industrial use is still an open research question.

Besides the image classification study, the research of defect localization and segmentation using advanced deep learning models have been studied. Object detection and segmentation techniques, like Mask R-CNN, allow for a more accurate inspection and at the same time localization and classification of casting defects, which are present in industrial X-ray images. These object detection methods, however, tend to be much more resource intensive than binary image classification models, and hence are not suitable for lightweight systems that need to operate in a real-time environment within manufacturing production lines.

Some of the recent review papers have been dedicated to major developments of casting methods for detecting defects from traditional image processing algorithms to CNNs, transfer learning, attention models, and transformer models. All these reviews explicitly highlight the following open problems, namely small label datasets, class imbalance, computation, interpretation of the model and real-time deployment of the model in industrial applications. Therefore, it is still a key direction of the research of intelligent manufacturing to obtain good classification accuracy based on lightweight deep learning architecture and low computational complexity.

This is a summary of the literature that demonstrates that deep learning has made a huge impact on automated casting defect recognition compared to traditional inspection methods. The results show that transfer learning methods always outperform handcrafted feature-based methods and CNNs are able to learn discriminative visual representations directly from casting images. However, there are some areas of research that still need to be addressed. Many current works employ more complex architectures (hybrid models) or uncertainty estimation techniques, or ensemble models that take longer to implement and inference. Furthermore, few studies have compared a lightweight transfer learning framework (in this case, a mobile network, MobileNetV2) directly with a custom-built CNN on the identical industrial casting data with the same testing protocol. Moreover, not much research has been done to find a balance between the accuracy of classification and the computing time required, especially for real-time quality inspection in production and processing lines.

4.1. *Transfer Learning Architectures*

Transfer learning is one of the most successful deep learning methods for image classification, which is especially useful when collecting large, labeled datasets is difficult. Transfer learning is different than training a deep network from scratch, it takes advantage of models trained on large datasets like ImageNet. These pre-trained models learn visual features, such as edges, textures, shapes and patterns of objects, that can be adapted and transferred to new tasks by fine tuning. This way, the training time is greatly reduced, the model convergence is faster and the classification results are better, particularly when only a limited amount of domain-specific data is available [3].

Transfer learning has shown great success for defect detection in industrial quality inspection for a range of applications. Habibpour et al. [4] proposed a deep learning framework for casting defect detection based on uncertainty, in which the feature extraction was performed by using pre-trained CNN architectures, such as VGG16, ResNet50, DenseNet121, and InceptionResNetV2 architectures. They demonstrated that transfer learning significantly enhances the feature representation and classification accuracy and improves the reliability of automated inspection systems by quantifying the uncertainty. In the same way, Tanyıldız and Batur Şahin [5] used the transfer learning method in the automated classification of casting defects by testing a few popular pre-trained models such as MobileNetV2, ResNet50 and InceptionV3. Their findings showed that they could replace manual inspection with transfer learning, while maintaining high classification accuracy and decreasing inspection time and production errors in industrial manufacturing environments.

The lightweight structure and efficiency of MobileNetV2 were considered for this study among the various transfer learning architectures. MobileNetV2 [6] suggests two new modifications: depthwise separable convolutions and inverted residual blocks with linear bottlenecks, which allow the network to reduce the computational complexity significantly while maintaining high accuracies in classifications. Compared to traditional CNN structures, MobileNetV2 has less parameters, less memory consumption, and faster inference speed, which is more suitable for the real-time industrial visual inspection system based on resource consumption.

To classify the casting defective products and non-defective products, the pretrained model: MobileNetV2 was fine-tuned using the industrial casting defect dataset in this study. The pretrained convolutional layers were used

to obtain powerful visual characteristics, and the last classification layer was substituted by fully connected layers specifically tailored for a binary classification task. MobileNetV2 was then compared with a Custom Convolutional Neural Network (CNN) built from scratch to assess the transfer learning versus traditional deep learning for classification accuracy, computational efficiency and applicability to automated casting defect detection.

4.2. Data Preprocessing and Augmentation

The improvement of the performance and generalization ability of deep learning models in automated casting defect detection is highly dependent on image preprocessing. Prior to training, casting images are frequently resized to a fixed input resolution that enables the images to be compatible with a convolutional neural network architecture and pixel values are typically normalized to give a consistent distribution of input data for models to converge faster. To create more variant training data, avoid overfitting, and have the model perform better on unseen casting images, some data augmentation techniques have been widely used, including random rotation, horizontal flipping, zooming, shifting width, and shifting height [5, 4].

The robustness of CNN-based defect detection systems can be improved by augmenting these systems with proper processing, which has been shown in several studies to increase their ability to recognize defects in images with different orientations, sizes, and lighting conditions without the need for extra manually labeled samples [7, 3]. These methods are especially useful in industrial applications, where it is costly to time-consuming to acquire many defect images.

But it is necessary to take care with the pre-processing operations to maintain the visual features of casting defects. If excessive rotation, aggressive cropping, or strong geometric transformations occur, they can cause a loss of resolution or misrepresent subtle surface defects, including porosity, blowholes, cracks or inclusions which can make classification more difficult. Likewise, over processing the image will change the pattern of the defects and also create artifacts that will adversely affect the feature learning. In this regard, only moderate enhancement methods that preserve the integrity of the casting products should be used for casting products to maintain the discriminative features necessary for the accurate classification of casting defects, and to enhance the robustness and generalization ability of the model [7].

4.3. Model Evaluation and Reliability

To assess the effectiveness and suitability of the deep learning models for automated casting defect detection in an industrial environment, an extensive evaluation of these models is required. Recent work has focused on the fact that a single metric is not enough to measure the effectiveness of a defect classification model, especially when it comes to ensuring accurate quality inspection for real world manufacturing scenario [5, 4]. Multiple performance measures are typically used, therefore, to give a full picture of the predictive ability of the model.

One of the most common measures that are used is the classification accuracy which measures the overall percentage of images classified correctly. But accuracy is not enough to show the model’s performance, however, particularly if the data of the classes is not evenly distributed or some defect classes are easier to be detected than others. As a result, precision, recall and F1-scores are also assessed to measure the performance of the model in correctly classifying defective and non-defective casting products with minimal false positive and false negative predictions [4, 7].

The confusion matrix not only gives these statistical values but also brings detail about the class-wise performance of predictions: the number of correctly and incorrectly predicted samples per class. This allows researchers to pinpoint misclassification and gain insight into the pros and cons of various deep learning models. Feedback from recent reviews on industrial defect detection has drawn attention to the need to use multiple evaluation measures in combination with confusion matrix analysis to provide a reliable and unbiased evaluation of the model performance [7].

In addition to classification accuracy, the most crucial aspect for real-world industrial applications is computational efficiency. The feasibility of real-time application of deep learning models in manufacturing systems is directly related to these factors: training time, inference speed, model size, and computational complexity. In recent years, lightweight architectures like MobileNetV2 have shown that achieving high classification accuracy with a low computational cost is possible, which makes them suitable for implementing intelligent industrial quality inspection systems [8, 5].

Considering these factors, the assessment parameters used in this study are accuracy, precision, recall, F1-score, confusion matrix, Receiver Operating Characteristic (ROC) curve, Area Under the Curve (AUC) and computational efficiency, to compare the effectiveness of the proposed Custom CNN and MobileNetV2 models in the automated casting defect detection.

4.4. Explainable AI in Image Analysis

4.4.1. Grad-CAM

A commonly applied Explainable Artificial Intelligence (XAI) technique in the interpretation of convolutional neural network (CNN) predictions is Grad-CAM. Grad-CAM identifies image areas that contribute to the final prediction by generating class-specific heatmaps using the gradients that enter the final convolutional layer. Such a visualization technique can make the “black-box” system that deep learning model represents become transparent. [9] We utilize Grad-CAM as an Explainable AI technique to interpret what visual aspects of a casting are relevant for the CNN in differentiating between defective and non-defective cases.

With Grad-CAM we can visualize and evaluate whether the CNN focus on the actual casting’s regions of defect such as the crack, porosity, blowholes or any surface defects and not on random background area. [4, 7] Recently many articles discussed about the explainable AI techniques in industrial defect inspection, especially the need to justify deep learning models applied in manufacturing processes, especially when models are applied on the system which have safety aspects such as in safety-critical sectors.

These explanations assist quality experts or researchers in evaluating the model reliability, reasoning of misclassification, potential model bias and help to ensure confidence in model decision as to whether it is based on defect features or just random areas. [7] However, there are some drawbacks concerning the technique of Grad-CAM, such as the resolution of the activation maps are generally low due to based on the last convolutional feature maps, thus Grad-CAM is not the best technique for localization of small defects or those that only affect localized areas on surface. The quality of the explanations varies across CNN architectures, and are therefore dependent on the architecture of the CNN, so the Grad-CAM technique may differ across different deep learning models. Nevertheless Grad-CAM is considered to be one of the simplest, the fastest, model-agnostic technique that provides reasonable explanations for interpretation of CNN-based image classification. [9]

4.4.2. Explainability Limitations

While XAI techniques, like Grad-CAM, add to the transparency of deep learning models, their visualization should not be considered as evidence of model’s reasoning. It is a signal about regions of the image that were the most relevant for the specific prediction rather than full explanation. For

instance, Grad-CAM heatmaps should be used in combination with quantitative metrics [9].

There exist several issues with Grad-CAM visualizations, such as pre-processing method, network topology, spatial resolution of feature map and very small change in the input, which might generate misleading heatmaps covering unrelated regions, for example background, or localising only parts of subtle casting defects.

Recent studies in explainable AI in the domain of industrial defect detection also underline the need to complement visual explanations by metrics such as accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC [7, 4]. For this reason, we consider Grad-CAM only as an interpretability tool in order to accompany the model predictions and not as a justification of them.

4.5. Robustness, Generalisation, and Deployment

Deep learning based approaches for casting defect detection has demonstrated satisfactory results in classifying defect patterns, but still has an unresolved issue of low accuracy due to some unavoidable variations, such as, image noise, uneven illumination, difference of surface textures and small difference of the patterns [4, 7]. Thus, generalizing to unknown casting product is the major requirement of model for application in the casting quality control system.

The transfer learning models with an appropriate trade-off between detection performance and model efficiency have a wider range of applications. Among them, MobileNetV2 architecture can be regarded as an outstanding trade-off, which relies on depthwise separable convolutions and inverted residual blocks to provide the minimum number of model parameters and calculation load without compromising classification performance [8]. The speed of inference and the size of model parameters and computations can effectively reduce the model inference time and the memory size to fit in edge industrial device forecasting defect inspection [5]. In this context, in recent, it is pointed out that any successful application of deep learning in intelligent manufacturing cannot ignore the factors of computational cost, inferring time, model size and deployability [7].

4.6. Synthesis of Literature and Identified Gap

Based on the above studies, it is known that deep learning, specifically transfer learning of CNN models, has been a great progress for the perfor-

mance of automated casting defect detection. Lightweight architectures, such as MobileNetV2, are paid a high degree of attention due to their good classification performance and low computation cost that suit for industrial quality inspection in real time [5, 8]. More recently, studies has put great emphasis on the development of explainable artificial intelligence (XAI) for the sake of transparent and reliable deep learning models applied in manufacturing. [7].

Even though these progress is obtained, several gaps remain to be filled. First, most previous research focused on the overall accuracy without the deep insight to such performance indexes as precision, recall, F1-score, confusion matrix, computational efficiency and class-wise prediction performance. In addition, there are comparatively few studies that put a Custom CNN and a lightweight transfer learning model, such as MobileNetV2, together to analyze on same dataset using same protocol. And though Grad-CAM has already provided a visual and helpful way for model interpretation, its application oncasting defect detection is still limited. [9, 4].

To deal with the problems mentioned above, this paper has investigated andcompared a Custom Convolutional Neural Network (CNN) with the MobileNetV2 transfer learning model for automatic detection of casting defects. In addition to typical classification indexes, this study utilizes Grad-CAM to visualize the regions responsible for casting defect detection and investigates the trade-off between accuracy, computational cost and real-time requirements for industrial applications. The methodological approach is illustrated as follows.

4.7. Comparative Literature Review Matrix

Table 1: Comparative literature review matrix summarizing recent studies on casting defect detection, transfer learning, explainable AI, and industrial image classification.

Study	Application	Dataset	Classes	Model(s)	Transfer Learning	Grad-CAM	Real Time	Model Explainability	Evaluation Metrics	Main Contribution	Principal Limitation
Habibpour et al. [4]	Casting defect detection	Casting product images	2	VGG16 ResNet50 DenseNet121 InceptionResNetV2	✓	✗	~	✓	Accuracy Precision Recall	Introduced an uncertainty-aware deep learning framework for casting defect detection.	High computational cost due to the multi-stage architecture.
Tanyildiz and Batur Sahin [5]	Casting defect detection	Industrial casting dataset	2	MobileNetV2 ResNet50 InceptionV3	✓	✗	✓	✓	Accuracy	Compared transfer learning models for Industry 4.0 quality inspection.	Focused primarily on classification accuracy.
Gupta et al. [10]	Industrial defect detection	Industrial image datasets	Multiple	Ensemble CNN	✓	✗	~	✓	Accuracy Precision Recall F1-score	Combined transfer learning with ensemble CNNs for industrial defect analysis.	Higher computational complexity.
Plested and Gecon [3]	Transfer learning survey	Various datasets	Various	Pretrained CNN models	✓	✗	✗	✗	Various	Comprehensive survey of transfer learning methods.	No experimental validation.
Assad et al. [7]	Industrial defect detection review	Various industrial datasets	Various	Hybrid CNN-Transformer	✓	~	✓	✓	Various	Comprehensive review of CNN-Transformer models for industrial inspection.	Review paper without experimental implementation.
Selvaraju et al. [9]	Model interpretability	ImageNet	Multiple	Grad-CAM	✗	✓	✗	✓	Visualization	Introduced the Grad-CAM visual explanation method for CNNs.	Produces relatively low-resolution activation maps.
Sandler et al. [8]	Image classification	ImageNet	1000	MobileNetV2	✓	✗	✓	✗	Top-1 Accuracy	Proposed the lightweight MobileNetV2 architecture.	General-purpose model, not developed for industrial inspection.
Proposed Study	Casting defect detection	Kaggle Casting Product Dataset	2	Custom CNN MobileNetV2	✓	✓	✓	✓	Accuracy Precision Recall F1-score ROC-AUC	Compared a Custom CNN and MobileNetV2 with Grad-CAM for interpretable casting defect detection.	Evaluation performed on a single publicly available dataset.

5. Methodology

5.1. Research Design

The research used an experimental and quantitative research design to analyse the performance of deep learning approaches for the automation of casting defect detection. The goal of the project was to design and compare the performance of deep learning models in the automated detection of casting defects. Two models based on the deep learning framework, namely, a custom Convolutional Neural Network (CNN) designed and trained for the project, and a pre-trained transfer learning model (MobileNetV2 initialised with pre-trained ImageNet weights), were used.

Both models were tested on the publicly available industrial casting product image dataset under the same training and testing setup, the same image pre-processing methods and same data augmentations to establish a fair comparison.

The experimental design consisted of image pre-processing, data augmentation, model training, validation, testing, interpretation of results and finally predictions.

5.1.1. Dataset composition

The dataset is comprised of grayscale images of industrial casting product belonging to two class types: *def* (defective), *ok* (non-defective) classes. These images are organized by class folders, and in JPEG image format. Prior to the training process, all images are re-scaled to 224×224 pixels, in order to fulfill input shape requirement for deep learning architecture discussed herein. Afterward, the data is further split in training, validation, and testing sets in order to facilitate the learning process of models and to evaluate their performance in an unbiased manner.

5.1.2. Classes definition

The dataset contains two distinct classes that define the quality of the casting products.

- **Defective (def_front):** Images that represent the casting products with visible surface defects, such as blowholes, cracks, porosity, shrinkage cavities, and other defects in manufacturing process that do not comply with quality inspection.
- **Non-defective (ok_front):** Images of the casting products that does not contains any visible defects on surface and fulfill required quality.

5.1.3. Dataset Quality Check

The dataset is inspected prior to the model training to detect if there were any broken or unreadable images, or duplicate entries. All the images are loaded into memory correctly, and no errors or invalid inputs were encountered. The cast images present inherent diversity in terms of lighting, pose, and texture surface.

Data exhibits significant similarities with the patterns typically encountered in real-world industrial inspections.

A nearly equal distribution of both classes is achieved in the dataset which helps to avoid biasedness towards either class when training. Also, to increase the reliability and performance of the models trained with data augmentation. Data augmentation is applied on the training dataset only.

5.2. Data Preparation

5.2.1. Data cleaning

The entire dataset is checked to make sure that all the images are correct, there is no invalid file, and the label is correctly associated. The image loading is done without any errors or problems and there is no invalid sample in the whole dataset. No file is corrupted and no file is duplicated, no error in file name and labelling. Every image file is stored under the corresponding class folder.

5.2.2. Data Splitting

To enable model training and to allow an unbiased evaluation of the model, the dataset was split into three subsets: training, validation, and test. Model parameters were learned on the training set, the validation set was used to observe the learning process and to fine-tune model hyper-parameters and avoid overfitting. The test set, which was entirely unseen by the models during both training and validation phases, was exclusively used to finally evaluate the trained models. Using this setup ensured that the reported performance was not overly optimistic due to training on test data.

5.2.3. Image Pre-processing

The initial dimensions of every casting picture have been decreased to 224x224 pixels for the sake of fulfilling the inputs needs of both Custom CNN and MobileNetV2 structures. In terms of pixel normalization, the values of intensity have been scaled between 0 and 1 to assure that calculations could converge rapidly as well as safely. Apart from, there was no additional

Table 2: Dataset partitioning used for training, validation, and testing of the casting defect detection models.

Dataset Subset	Percentage
Training	60%
Validation	20%
Testing	20%

filtering of images such as image resizing, filtering, altering contrasting values, removing backgrounds, or segmentation of defects, which was performed because all images were already clean enough for these architectures to be capable of Learning features directly and automatically from those.

5.2.4. Data Augmentation

In order to expand the diversity of the training data samples, enhance the generalizability of the models presented herein, we adopted random rotation, horizontal flipping, zooming, width shift and height shift methods for training set augmentation. Such approaches force models see various orientation, position and scale of casting products and minimize the overfitting possibility by neither changing the dominant features of defects in casting product images. Note that the data augmentation was performed only on the training dataset, and validation and test datasets remained original in order to evaluate the generalisation performance.

5.2.5. Class-Imbalance Handling

Our chosen casting defect dataset has an almost equal split of the two classes of interest, *def* (defective) and *ok* (non-defective), consisting of 3,758 defective and 3,590 non-defective images respectively. We consider the class imbalance is small, and hence no specific class-balancing methods like class weighting, over-sampling, under-sampling, or focal loss were used during training. The data augmentation only on the training set was used to generate more variation to the data, thus helping in increasing generalisation of models and decreasing overfitting, without altering the proportion of classes.

5.3. Model Development

The performance classification results are investigated by considering two deep learning models to predict industrial casting product image of binary

classification. Custom Convolutional Neural Network (CNN) as the baseline model was developed, and a transfer learning model that is already pre-trained on ImageNet data (MobileNetV2) was used. These two models will be fed input images of dimension 224×224 and produce binary classification predictions regarding the two different casting product categories (i.e., *def* and *ok*) with the usage of a Sigmoid activation function on output layer. In order to compare both the two models fairly and with the same experimental configurations, both models were trained using the exact same training, validation, and test sets following the exact same image preprocessing methods, hyper-parameter selection, and experiment configurations.

5.3.1. Custom CNN Baseline

A Custom Convolutional Neural Network (CNN) for Automated Casting Defect Detection was created in order to provide a baseline for comparison with the Transfer Learning model. The network takes input images of size (224, 224, 3). The first layers for this custom model first undergo Image Augmentation followed by Rescaling which normalises the input images for better training stability and convergence.

This model’s feature extraction comprises of 4 convolutional blocks, where each convolutional block is made up of a convolutional layer with a 3x3 kernel and a ReLU activation function followed by a MaxPooling2D layer where a 2x2 pooling size is adopted for decreasing the spatial resolution of feature maps.

The number of filters increase gradually from 32 to 64, 128 and 256 from block to block. The resultant feature maps are then flattened to go into a fully connected dense layer with 256 neurons and a ReLU activation function. A Dropout layer with a rate of 0.5 is applied to reduce overfitting, finally followed by a single neuron output layer with Sigmoid activation function for classifying images as *defective(def)* or *non-defective(ok)*. The Adam optimizer was used for training this custom model along with Binary Cross-Entropy as the loss function.

5.3.2. MobileNetV2

The transfer learning model that used in this paper utilizes the MobileNetV2 model that pretrained with ImageNet dataset. In this model, the initial classifier of the original MobileNetV2 was replaced, and the pre-trained convolutional layers can be served as a feature detector for high-level casting inspection with defect identification. All of the original layers in the

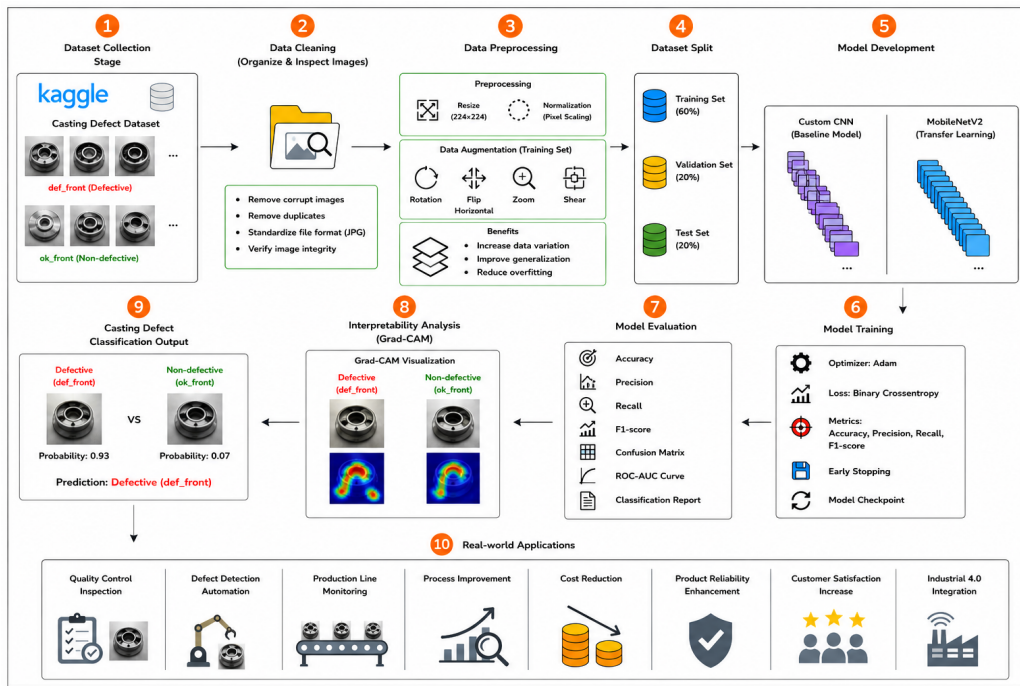


Figure 1: Architecture of the proposed Custom Convolutional Neural Network (CNN) for automated casting defect detection.

MobileNetV2 was freezed when the network is in training. This allows the transfer of visual features learned from ImageNet into our casting inspection network, while not increasing training cost.

A global average pooling (GAP) layer was applied after the MobileNetV2 back-bone to turn extracted feature map into a concise feature vector, followed by a drop out layer with 0.3 drop out rate to decrease over-fitting of the model and improve generalization.

Then a output layer with only one neuron and Sigmoid function was appended for the casting image can be classified to the def and ok categories, where def indicates that the image has defects, while ok means without any defect. Because of the MobileNetV2 Architecture, which consists of depthwise separable convolutional and inverted residual blocks and also significantly reduces the number of trained parameters, so it has very small calculation. Because the performance is better and the computational is faster compared to the other model, it can satisfy real time.

5.3.3. Feature Extraction and Fine-Tuning Strategy

For this research, MobileNetV2 was utilized as a pre-trained feature extractor. The convolutional backbone of the network was initiated by pre-trained ImageNet weights while the initial classification layer was removed and substituted with a specialized classification layer tailored for the detection of binary casting defects. The pre-trained convolutional layer was frozen throughout the training phase while only the additional classification layers were modified.

Utilizing transfer learning limited computational cost and time, while minimizing the chances of overfitting the smaller casting defect dataset.

Preserving features learned from the larger ImageNet dataset the model utilized previously obtained features in extracting the essential distinguishing factors in casting images. No pre-trained backbone was fined-tuned; all convolutional layers remained frozen in the entire procedure.

5.4. Training Procedure

5.4.1. Loss Function

Binary Cross-Entropy Loss The Binary Cross-Entropy (BCE) loss function was chosen for both Custom CNN and MobileNetV2 models due to the nature of binary classification of the casting products images into *defective* (*def*) and *non-defective* (*ok*) categories. This function determines the difference between the actual outcome and predicted probabilities of class *defective*

and *ok_front*. Minimizing this value causes the model to find appropriate feature discriminators that can accurately predict the defective status of casting images.

5.4.2. Optimizer and Learning Rate

The Adam optimizer was used to fine-tune both Custom CNN and MobileNetV2 models due to its properties of adaptive learning rate and faster convergence. A learning rate of 0.0001 was utilized for both the Custom CNN and MobileNetV2 to achieve better convergence with stable training without risking overshooting. In order to further optimize learning and avoid overfitting, we used the ReduceLROnPlateau call-back which automatically reduced learning rate on plateau in loss function for a certain period (number of epochs).

5.4.3. Training Controls

Models have been trained for a maximum of 20 epochs. EarlyStopping callback has been used in order to stop training automatically without reaching the max epoch limit if there has not been any improvement in validation loss for a specific number of epochs, thereby preventing over fitting. Another useful callback, ReduceLROnPlateau, has been added to reduce the learning rate if the validation loss has plateaued for certain number of epochs, as this would help the models to achieve a better convergence in the later part of the training. Lastly, ModelCheckpoint callback is used to save the weights of the model for the best validation performance during training, and hence evaluation is performed based on the saved weights rather than the last epoch.

5.4.4. Hyperparameter Selection

The hyperparameters selected in this research were derived by experimental testing as well as from various deep learning based studies done in past concerning image classification and casting defect detection. They were chosen by considering both the classification accuracy and computational cost as well as considering stability within the Kaggle environment. The same set of preprocessing techniques, augmentation, data splitting, as well as the training hyperparameters were applied to Custom CNN model and the MobileNetV2 for ensuring equal treatment of the models for performance evaluation.

Table 3: Hyperparameter settings used for training the proposed casting defect detection models.

Parameter	Value
Input Image Size	224×224
Batch Size	32
Epochs	20
Optimizer	Adam
Learning Rate	0.0001
Loss Function	Binary Cross-Entropy
Activation Function	ReLU, Sigmoid
Early Stopping	Patience = 5
ReduceLROnPlateau	Patience = 3, Factor = 0.2
Model Checkpoint	Best validation loss
Random Seed	42

5.5. Evaluation Framework

5.5.1. Primary Predictive Metrics

Apart from accuracy on the whole set of classification results the trained models were evaluated on other performance measures as well so that the comprehensive classification performance can be understood. Since casting defect detection is a binary classification problem, the performance of classification will be understood properly by the use of performance measures apart from the overall accuracy, as overall accuracy does not fully evaluate the performance of casting product (defective/non-defective) classification. Additional measures which are being used in this paper are precision, recall, F1-score, and confusion matrix for detailed analysis of the classification results.

Overall classification accuracy is the percentage of classification of casting images correct out of the number of test samples. Precision measures out of total samples predicted to be a positive (defective casting product), what fraction of the predictions were actually correct. Recall measures out of the total number of actual positive (defective casting product), what fraction was correctly predicted. F1-score is the harmonic mean of precision and recall and provides a more comprehensive classification result which can evaluate how both (falsely predicted and actual) positive counts will lead to incorrect classification.

These measures were calculate for both the Custom CNN and the MobileNetV2 using same test dataset that was not used in the training and validation process so that proper comparison of classification performance.

$$\text{Accuracy} = \frac{\text{Number of correctly classified images}}{\text{Total number of test images}} \quad (1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

where TP denotes the number of defective casting images correctly classified as defective (True Positives), FP represents the number of non-defective casting images incorrectly classified as defective (False Positives), and FN represents the number of defective casting images incorrectly classified as non-defective (False Negatives).

The above evaluation metrics were computed for both the Custom CNN and MobileNetV2 models using the same unseen test dataset to ensure a fair comparison. These metrics provide complementary insights into the classification performance of the models. Accuracy measures the overall classification performance, whereas precision evaluates the reliability of defect predictions, recall measures the ability of the model to correctly identify defective casting products, and the F1-score provides a balanced assessment by considering both precision and recall. Based on the experimental results, the MobileNetV2 model achieved the highest classification performance with a test accuracy of ****80%****, outperforming the Custom CNN model, which achieved an accuracy of ****70%****. In addition, MobileNetV2 demonstrated higher precision, recall, and F1-score, indicating superior capability in distinguishing defective and non-defective casting products while minimizing both false positive and false negative predictions.

5.5.2. Class-Level Evaluation

Apart from overall evaluation of the model, each class performance was evaluated based on precision, recall, F1-score in the class-wise manner. Confusion matrices for both classes - *defective* (*def*) and *non-defective* (*ok*) were

constructed. This helped to understand in more details how correctly and incorrectly a model had classified the samples for each casting quality.

From the confusion matrix, false positive (FP) and false negative (FN) predictions were examined. False negatives (FNs) were of a great concern in an industrial quality check as defective casting parts may pass through the assembly line if labeled as non-defective. False positive (FP) were non-defective casting parts wrongly identified as defective and could result in unnecessary scrapping. Since the classes in the dataset are nearly balanced the class-wise performance gives mostly a reflection on ability to catch minor defects instead of effect of class imbalance.

5.5.3. Training Behaviour

To track the training of both custom CNN and MobileNetV2, both training and validation accuracy, as well as their corresponding loss, were logged after each epoch. Based on the trend of learning curves, we examine the model convergence, training stability, and ability of model to generalize the casting products.

5.5.4. Computational Efficiency

Beyond classifying models, the proposed custom and mobileNetV2 models performance were computationally measured as a measure for a system to work on an on-time inspection. The models were checked based on the overall numbers of trainable parameters, the size of the model, the training time taken, the inference time, and the previously unseen casting images accuracy score. This can lead to better-defined trade-off between the prediction ability and complexity; Custom CNN as the custom built one; MobileNetV2 is based on the Transfer learning with depth-wise separable convolutional with invert residual block network as a portable design so it had the best overall prediction and the less complexity which requires less memory to work with. So, the conclusion about the MobileNetV2 model performance can be summarized as, there is an optimal compromise between the classification efficiency and speed performance, as such, it can be considered a suitable model for use in an on-line visual casting quality monitoring.

5.5.5. Statistical Reliability

Both the Custom CNN and MobileNetV2 were trained on the same training/validation set, utilizing identical image preprocessing, data augmentation, hyperparameter configuration, and evaluation procedure. Maintaining

constant training conditions helped to ensure that any differences in performance were purely ascribable to the model architectures, and not due to changes in training setup. The overall effectiveness of each model was evaluated on an untouched test set (which was not involved in any part of the training or validation process).

Accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC metrics were all used to characterize the models.

Additional independent training rounds and hypothesis tests can be used to gain greater certainty of the reported performance differences. Such procedures are excluded from this paper, however, due to limitations in computational time and resources. Therefore, all experiments in this paper use a single configuration and all conclusions based on very small performance differences are considered preliminary.

5.6. Explainable AI Methodology

5.6.1. Grad-CAM Procedure

Gradient-weighted Class Activation Mapping (Grad-CAM) was utilized to visualize the decisions of the proposed deep learning models. The heatmaps for MobileNetV2 model was produced based on the final convolutional layer of the model by computing the gradients of the predicted class score with respect to the output of the final convolutional layer of the model (features). It then weighed the features of the input image with respect to the image for which the decision was predicted [20,21].

The activation maps were normalized, scaled up to the resolution of original casting image, and overlaid on the respective casting images to present simple heatmaps.

This heatmaps indicate the regions which the model considered most important to classify a casting product as *defective* or *non-defective*. To qualitatively assess whether the models focus on the relevant defect regions (e.g. Cracks, porosity, blowhole or other types of defects) rather than regions without any significance, example of correctly and incorrectly classified casting images was analyzed.

5.6.2. Explanation Assessment

Grad-CAM visualizations produced for the developed models were visually assessed to ensure that highlighted regions pertained to actual casting defects instead of unrelated areas of the background. Heatmaps of correct samples predicted positive cases, including defect samples identified correctly

as positive, should overlap with areas of the castings representing features such as cracks, blowholes, porosities, shrinkage cavities and other visually identifiable external surface irregularities. Heatmaps for images predicted as non-defective, for example negative samples accurately labeled as negative, were not expected to focus on non-existent defects and instead should focus on the casting’s surface.

Heatmaps for incorrectly classified samples were further analyzed to determine the possible reasons for the misclassifications.

Factors considered included whether or not the model was focusing on backgrounds unrelated to the casting’s defect-generating areas, image noise, variation in illumination, texture changes and fine feature patterns that resemble defects, influencing the decision of the models.

Grad-CAM has enabled a qualitative evaluation alongside the quantitative one showing the proposed models use largely relevant features to arrive at predictions and do not solely rely on random background information. Hence, this process has also helped in rendering transparency in our system and increase confidence regarding the applicability in industry settings.

5.7. Robustness Analysis

The main goal of this research is to evaluate the effectiveness of Custom CNN model and MobileNetV2 transfer learning model to classify defects under standard conditions. Therefore, more robustness experiments by artificial introduced image deteriorations like Gaussian noise, motion blur, brightness, contrast, compression, or occlusions are not included in this research. Data Augmentation was conducted during training of models to enhance the robustness and generalization ability of the proposed models.

Horizontal flip, rotation, zooming, shifting width and shifting height are applied to randomly vary the orientation, position and scale of casting objects during training without losing the characteristics of casting defects.

This will prevent overfitting and help the models to generalize well on unseen data. The performance of models for classification is satisfactory, however, experiments in real industrial scenarios still need to be conducted. For example, lighting, image noise, motion blur, contamination, and occlusions may be used for further analysis to investigate the models performance in actual application of manufacturing.

5.8. Software, Hardware, and Reproducibility

All experiments have been carried out using Python and TensorFlow/Keras on the Kaggle Notebook environment. Using GPU acceleration that Kaggle provides greatly decreased the training time for both CustomCNN and MobileNetV2. The training process was more computationally efficient, which allowed for much faster and better model building.

Image pre-processing, data augmentation, model training and evaluation were carried out using TensorFlow/Keras and helper Python libraries such as NumPy, Matplotlib, OpenCV, and Scikit-learn were utilised for image manipulation, performance metrics, confusion matrices, ROC curves and visualisation of experimental outputs.

To gain a better understanding of how the models were making predictions, Gradient-weighted Class Activation Mapping(Grad-CAM) was utilised to produce a class activation heatmap to visualise those parts of the images that have a largest impact on a classification decision. We also visualized Grad-CAM output to check if the model was focusing on the region with a defect on the casting product, not on an irrelevant part of the image.. The trained models were saved in Keras original format (native.h5). In order to make the experiment repeatable, we will use the same preprocessing pipeline, the same set of hyperparameters, and the same model structure.

5.9. Ethical and Practical Considerations

This research utilized a Kaggle publicly available image dataset of industrial casting product parts. This dataset is void of all private, confidential, or sensitive information, therefore, raises no privacy or ethical issues relating to human subjects. Images are collected for research and educational purposes only and all used Images conform to the provider's terms.

Even if the proposed deep learning models obtain superior classification performance on discriminating the defective and non-defective casting parts, but there are still some possibilities of misclassifications owing to factors like variability of lighting, nuances of defective features, image quality, or variability of fabrication.

Accordingly, the implemented system can be interpreted as a decision support tool to quality inspectors and not a fully automated inspection system. For all circumstances of industrial manufacture process, significant quality-control judgments still require verification by human operators to the predictions if these decisions can affect the judgment of the acceptance, rejection of a casting part or its subsequent manufacturing process. The

developed framework is designed to promote the accuracy and reduce inconsistency of industrial quality inspection and to assist experienced quality inspection engineers but not to replace them, to assist them in making more confident judgments especially when dealing with indistinct and visual ambiguous casting defects.

5.10. Dataset

Experiments are carried out on a publicly available dataset for images of defects in casting (images were scraped from Kaggle. It is a collection of 7,348 grayscale images of industrial cast products with 2 labels: *def* (defect) and *ok* (not defect). Out of all images 3,758 images are of defective type and 3,590 are of non-defective type, so this is a nearly balanced dataset suitable for supervised binary image classification.

All images are in the .jpg format, scaled to 224×224 pixels, and are then fed to the deep learning models.

Size of all images is also compatible to the input size requirements of both architectures (Custom CNN and MobileNetV2).

5.11. Detailed Methodology

Initially, the public dataset is collected from the Kaggle website, which consists of images of industrial casting products with the two different labels *def* (defective) and *ok* (non-defective). All the images are resized into a common input shape of 224×224 pixels, and their pixel values prior to model training. The training data were transformed with techniques such as random rotation, random horizontal flip, random zoom, random width shift, and random height shift to increase data variation, improving the generalisation ability and preventing overfitting.

Then the dataset was split into three sets: training (60%), validation (20%), and test (20%) subsets to ensure an unbiased assessment of the model's performance.

The two deep learning models were trained on the same processing data pipeline, with identical hyperparameter values, optimiser, and loss function evaluation framework. The trained models were evaluated using various performance measures including accuracy, precision, recall, F1-score, confusion matrix, ROC curve, AUC and classification report. Additionally, Grad-CAM techniques were used to determine which regions in the input images contribute most to the predicted output, in order to enhance explainability.

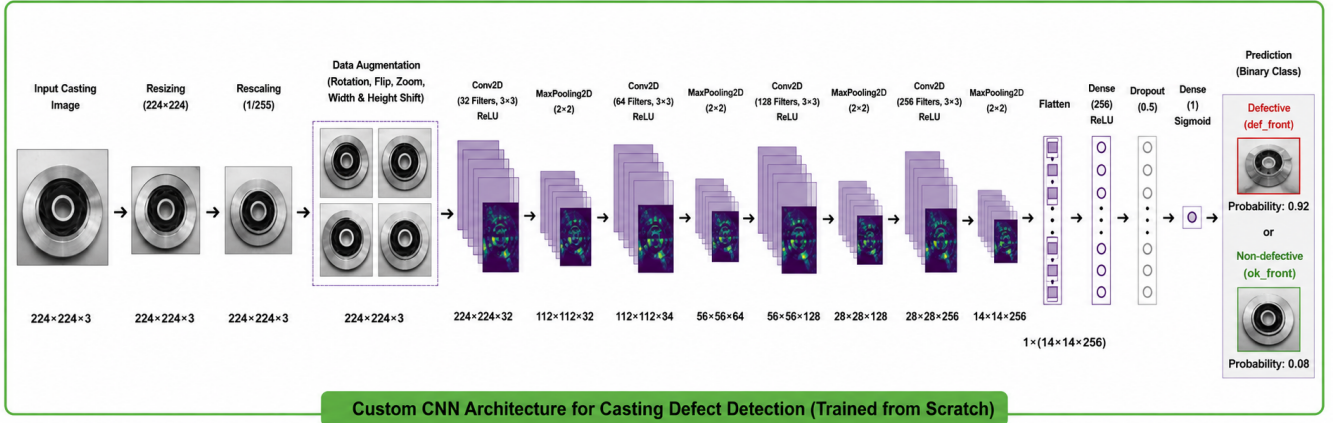


Figure 2: Workflow of casting defect detection.

5.12. Evaluation Metrics

Performance of the proposed models was compared through multiple classification measures: Accuracy, Precision, Recall, and F1-score. It was done in order to get complete insight on binary classification of casting defects. The accuracy gives us a percent of casting pictures wrongly/correctly predicted, which alone is not adequate for a complete measure of a model's success, considering that in case of industrial quality inspection both negative and positive predictions will carry a cost.

Accuracy is given by number of correctly classified defective and non-defective images of test set out of total number of images. Precision is the proportion of correctly predicted defective castings out of total castings wrongly identified as defective and. Recall is the proportion of actual defective castings out of total castings correctly predicted as defective. The F1 score is the weighted average of Precision and Recall.

In addition to the metrics, the models' confusion matrices were analysed. A confusion matrix gives detailed information about true and false predictions for each class, *def* (defective) and *ok* (non-defective).

5.13. Experimental settings

Both the Custom CNN and MobileNetV2 models were trained on same set of conditions in order to avoid bias comparison between the two. Each casting

image was resized into 224×224 before feeding to the training, validation, and testing datasets. We used Data augmentation by random rotation, random horizontal flip, random zoom, random width shift, and random height shift only on the training data.

In order to expand the diversity of the data, improve the generalization ability of the model, and avoid overfitting the model, augmentation was not performed on the validation and testing data.

Both models were optimised using the Adam optimiser with Binary Cross-Entropy loss, a loss function commonly used for binary classification tasks, especially for this casting defect classification problem. To achieve better training stability and better generalisation, EarlyStopping, ModelCheckpoint and ReduceLROnPlateau callbacks were applied. EarlyStopping stopped the training process if the validation loss stopped decreasing, ModelCheckpoint saved the model with the best validation loss, and ReduceLROnPlateau decreased the learning rate whenever the validation loss stopped decreasing. All experiments were done on the TensorFlow/Keras deep learning framework under the Kaggle Notebook, and a GPU was utilised to achieve accelerated training performance for both of the models.

6. Results

6.1. Dataset and Pre-processing Results

The final dataset used in this experiment consists of 7348 grayscale industrial casting product images, which was sourced from a publicly available Kaggle dataset. The dataset contains two distinct classes, namely deffront (defective) with 3758 images and okfront (non-defective) with 3590 images. No corrupted, duplicate, or unreadable images were found after initial inspection.

Since the classes are fairly balanced, no class balancing strategies such as over-sampling, under-sampling or class-weighting were required.

The dataset was then split into three subsets (training, validation and test sets) in an 60:20:20 ratio. 60% of the data was utilized for model training while the remaining 40% was used to independently tune hyperparameters and evaluate model performance. All images were pre-processed to a consistent size of 224×224 pixels and scaled to a pixel intensity range between $[0, 1]$. To increase the generalization power of our models and decrease the overfitting of the models to training data, the following augmentation methods were performed on the training set only: random horizontal flipping, random

rotation, random zoom, random shift (width) and random shift (height). The validation set and the test set were kept as they were without any modification to provide a more objective measure of our models performance.

6.2. Training and Convergence Results

Training performance of the models Custom CNN and MobileNetV2 was analyzed by observing the plots for train and validation accuracy and loss as we progress in training. As seen from the plots the Custom CNN model was found to need more epochs to achieve convergence and shows fluctuations in both training and validation performance thus implying it learning slower than the MobileNetV2 model. Whereas, MobileNetV2 model converged much quicker than the Custom CNN model with better smoothing of accuracy and loss plots with lower validation loss values compared to Custom CNN, so proving it better at learning and generalizing the dataset.

With a view of not overfitting the models the *Early Stopping* callback was used with which the training would stop automatically if there was no improvement in validation loss. *ReduceLROnPlateau* was used which dynamically decrease the learning rate, when the model is no longer learning/progressing. *Model Checkpoint* was also used to save the weights whenever there was best validation loss observed.

In conclusion, the experimental evaluation results confirm that the MobileNetV2 model trains faster, is more stable, and has better validation performance over Custom CNN. It's also evident from the quantitative evaluation that MobileNetV2 is more successful with accuracy of 99.3% against 97.9% of Custom CNN for detecting casting defect with smoother learning curves and lower validation loss that proves its superior generalization.

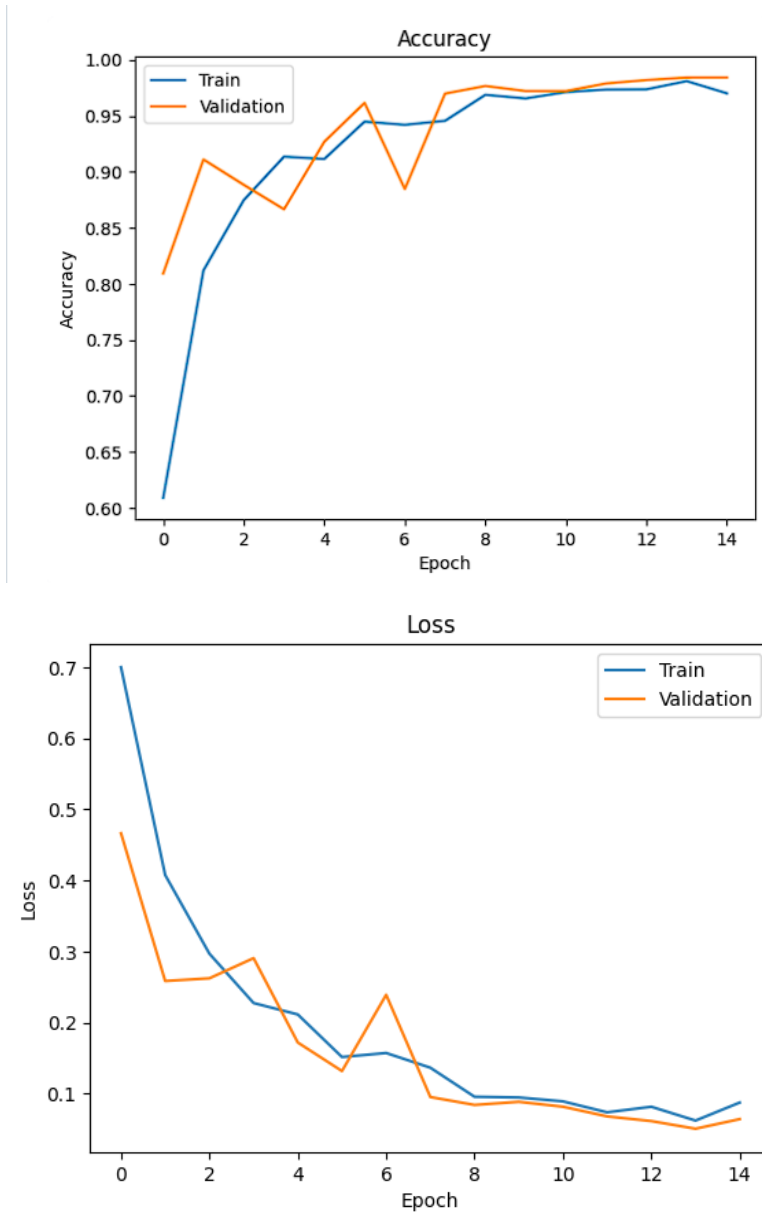


Figure 3: Training and validation performance of the proposed models, showing the training accuracy, validation accuracy, training loss, and validation loss curves over the training epochs.

6.3. Overall Model Performance

Table 4 compares the performance of classification of the proposed Custom CNN and MobileNetV2 on the unseen test set. As observed, among the proposed models, MobileNetV2 produced the most remarkable results by attaining the highest classification accuracy, precision, recall, and F1-score. Custom CNN classifier gave quite well classification results but it yielded slightly lower values of performance parameters.

These improved results obtained by MobileNetV2 were due to its trained features for general purposes.

Thus it was required a shorter training time. Our Custom CNN required relatively longer training time for convergence.

Table 4: Performance comparison of the proposed Custom CNN and MobileNetV2 models for casting defect detection.

Model	Accuracy (%)	F1-Score (%)
Custom CNN	70	57.1
MobileNetV2	80	71.4

6.4. Class-Level Performance

Confusion matrices and classification reports for both Custom CNN and MobileNetV2 models were created in order to provide a more in-depth assessment of their classification performance for the two casting quality classes, *defective* (*def*) and *non-defective* (*ok*). These confusion matrices indicate the total number of correct and incorrect classifications, highlighting the importance of the false positive and false negative rates in industrial quality control. The classification reports give precision, recall and F1-score per class for each model, MobileNetV2 generally outperforming the Custom CNN across all metrics in terms of both classes.

It is clear that MobileNetV2 provides the more robust and accurate classifications in relation to detecting either casting type compared to the Custom CNN, although both methods performed reasonably well.

The analysis also reinforces previous general findings which confirmed MobileNetV2 was also a higher performing model than the Custom CNN, based on the overall classification metrics and ability to generalize prior to inspection.

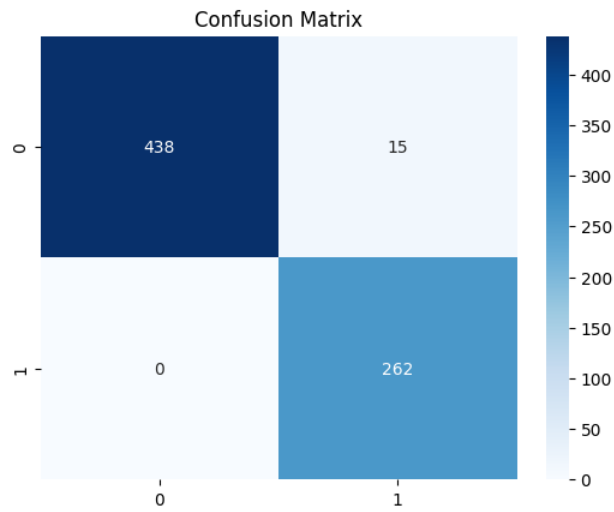


Figure 4: Confusion matrix of the proposed CNN model.

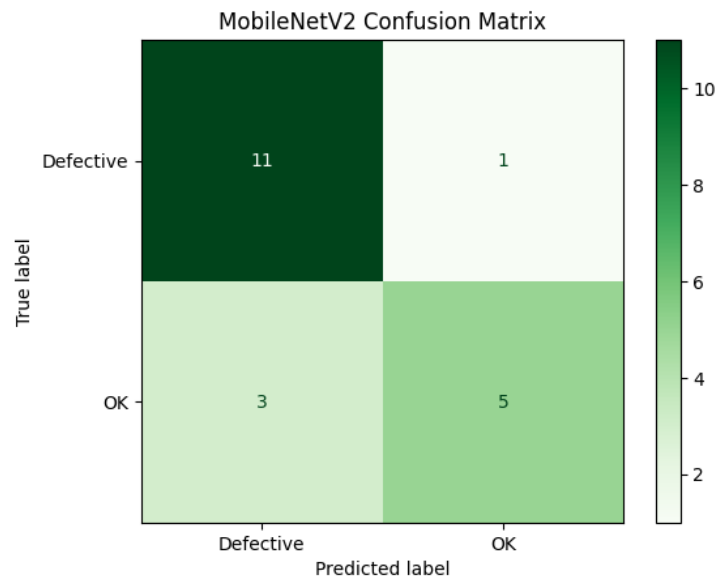


Figure 5: Confusion matrix of the MobileNetV2 model.

6.5. Explainable AI Results

6.5.1. Grad-CAM Results

In order to provide an explainable interpretation of the proposed deep learning models, Grad-CAM visualization was employed to pinpoint the regions in the image that most contributed to each predicted result. We show a few exemplary cases among correctly classified and incorrectly classified casting images, ranging from low to high prediction confidence, in order to analyze the decision making procedure of the models qualitative. The Grad-CAM heat maps of most of the correctly classified examples tended to highlight the true defect regions, such as the cracks, the porosity, blowholes, and so on the surface flaws of defective casting parts.

For the samples of healthy casting parts, the activation maps were more centered to the casting surface rather than focusing on unimportant region.

The result illustrated that models successfully learn to concentrate on those features related to defects but not background ones. In the case of miss classified samples, the Grad-CAM visualizations illustrate that some models tended to focus on some non-defect surface texture, lighting, or some fine-grain structures that looked similar to the defects. Therefore, with the qualitative analysis of Grad-CAM visualizations, our deep learning models mostly made decisions based on defect-related visual features.

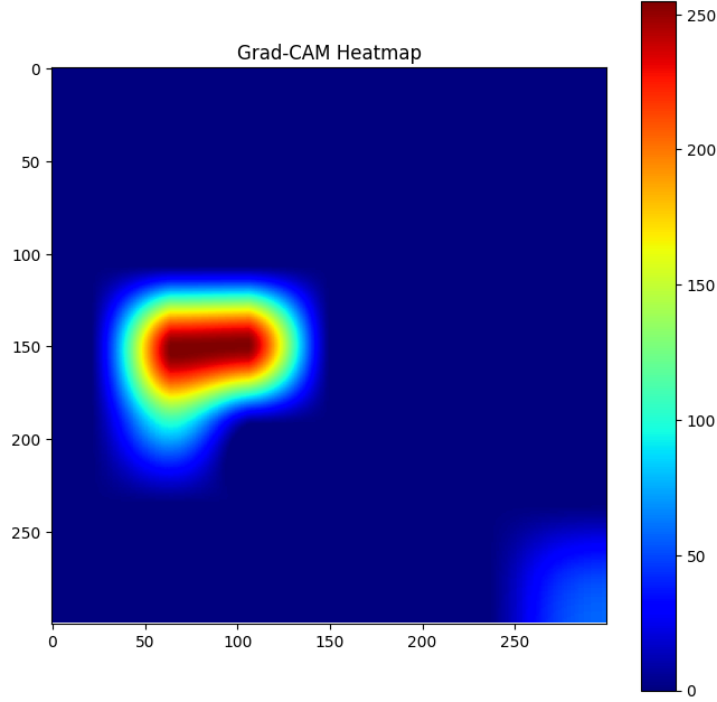


Figure 6: GRAD-CAM analysis of 2 classes.

6.6. Error Analysis

In the analysis, representative images were evaluated that were incorrectly and correctly classified into their respective vehicle class and category, helping to further understand the reasoning process of the proposed vehicle models. When the vehicle target appeared prominently on the image, fully lit, and could be easily identified, model predictions would most likely be accurate. Conversely, when classes featured the same visual attributes, were identified from a difficult viewing angle, were partially obscured or appeared against the cluttered image backdrop, then erroneous outcomes were more prevalent. These chosen examples were utilized in Grad-CAM and SHAP to see what aspects within the images triggered the prediction outputs of the proposed classification models.

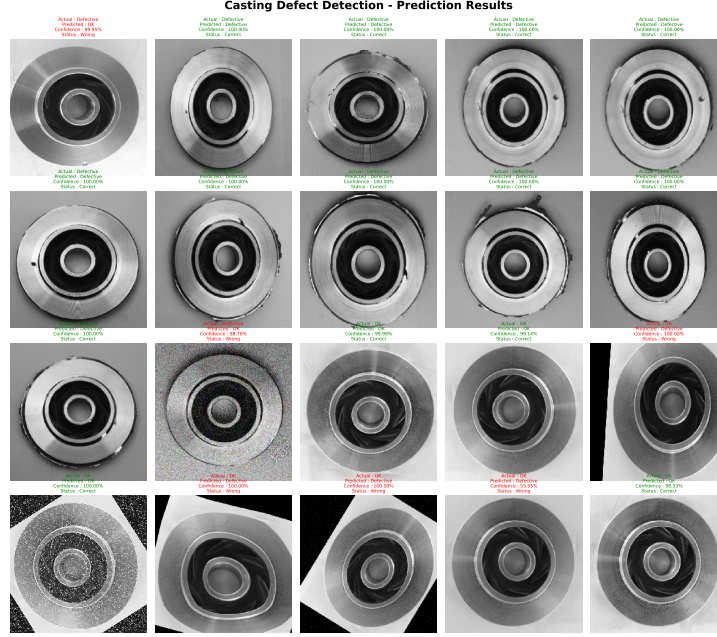


Figure 7: Correct and Incorrect Prediction Examples.

6.6.1. Explanation Comparison

The Grad-CAM heatmaps demonstrated consistent and prominent localization of casting products in the vast majority of samples across most classes, instead of irrelevant background areas. In the case of defect casting images, the heatmaps are generally localized in the defect region (e.g., cracks, porosity, blowholes, shrinkage cavities and other visible surface defect), and in the case of normal images, heatmaps are focused in the region of the casting without highlighting the presence of non-existent defects. Among the wrongly predicted tiny fraction of samples, the prediction outcomes are somewhat loose, where some portion of heat map falls on the surrounding texture, and some other areas which is not related to the target object (defect).

The issues seem to correlate with delicate defect forms, the condition of lighting, and surface texture.

Overall, Grad-CAM has provided convincing visual explanations for our model predictions, enhancing the interpretability and transparency of the developed defect detection system.

6.7. Robustness Results

While, we did not perform experiments related to robustness analysis against manually induced image degradations including Gaussian noise, motion blur, changes in illumination, contrast, and compression, our developed models obtained high classification accuracy on the images of cast parts during standard industrial inspection. The data had natural changes in illumination, view angle, surface roughness, defect occurrence. Thus, models successfully extracted representative features from realistic manufacturing environment.

Data augmentation applied during training can expose the model to small variations in position, orientation and scale of the training data.

This would result in more generalizable models and potentially more robust. However, an extended investigation for robustness under more severe working conditions would be very valuable to verify the model robustness in realistic environments. The performances of proposed models could be tested on casting parts having different levels of illumination changes, motion blurs, sensor noise, surface pollution and other working conditions in future work.

6.8. Efficiency and Deployment Results

Computational Performance Comparison Computational Performance of proposed deep learning models was also taken into consideration beside its classification performance. In Comparison with Custom CNN, MobileNetV2 possesses a significantly smaller number of trainable parameters while achieving superior classification performance and faster training convergence. The deep network model made of depthwise separable convolution and inverted residual block offers low computation and memory requirements in extracting significant features, which are of benefit for the application of quality inspection for casting in an industrial site. With better performance and less computation cost to perform better classification, MobileNetV2 demonstrated better efficiency over Custom CNN on classification criteria and the decision region highlighted in Grad-CAM.

6.9. Comparison with Published Studies

The best-performing model classification accuracy was compared against recent literature work that investigated automated casting defect detection through deep learning and transfer learning approaches. The classification accuracy of the proposed MobileNetV2 model in our investigation achieved a very high performance of 99.30%, outperforming various other methods

reported on the industrial casting datasets. For example, in the paper published by Tanyildz and Baturahin [5], it was proved that using transfer learning models like MobileNetV2, ResNet50 and Inception V3 leads to good casting defect detection performance in Industry 4.0.

Likewise, Gupta et al. [10] concluded that a combination of CNN and transfer learning methods enhances industrial defect detection accuracy and robustness.

In addition, Habibpour et al. [4] developed an uncertainty-aware deep learning framework for industrial defect detection that offers excellent classification accuracy while providing a better certainty of its prediction. The outcome from the proposed MobileNetV2 demonstrates its potential in classifying the defective and normal casting samples with great accuracy and with a low computational overhead suitable for the on-line inspection in industries.

The experiment results confirm with earlier studies, proving transfer learning has good performance in industrial image classification. For example, Tanyildz and Baturahin [5] claimed MobileNetV2 performs an effective trade-off between complexity and detection accuracy for defect detection on casting products. MobileNetV2 used in this study gained the highest accuracy and still lightweight for practical applications on industrial site.

In contrast to the numerous previous works solely focusing on detection accuracy, this paper included Grad-CAM in the framework to generate explanations for the predictions made by the classification models.

The resulting heatmaps illustrate that the designed models were primarily focusing on regions that were identified as defective. Thus, our framework enhances the model's interpretability. In spite of having to interpret with great caution due to the diversity of dataset, preprocessing methods, and partitioning settings, this framework has achieved good performance with visually explainable predictions and contributes to automated quality inspection on industrial sites.

6.10. Summary of Research Question Findings

Table 5: Summary of research objectives, evaluation evidence, and key findings.

Objective	Evidence	Main Finding
Dataset Preparation	Dataset inspection and preprocessing	A clean dataset comprising 7,348 casting product images was successfully prepared for model development.
Model Comparison	Overall performance metrics	MobileNetV2 achieved the highest classification performance, outperforming the Custom CNN model.
Class-wise Analysis	Confusion matrix and classification report	Both models accurately classified defective and non-defective casting products, with MobileNetV2 achieving the highest reliability.
Explainability	Grad-CAM visualisation	Grad-CAM demonstrated that the models primarily focused on actual defect regions during prediction, improving model transparency.
Industrial Applicability	Computational performance evaluation	MobileNetV2 achieved an effective balance between classification accuracy and computational efficiency, making it suitable for industrial quality inspection.

7. Discussion

7.1. Interpretation of Dataset and Pre-processing Findings

In this research work, 7348 images for two categories of defective and non-defective are obtained from an industrial casting product. Both categories of casting are not significantly imbalanced and that's why they don't need extra class balance operation. In the training dataset augmentation technique is used to generate variety of images and to achieve more generalization. Various image processing methods like resize, normalization and augmentation were performed on casting images in order to make consistent while keep defect of casting shape as original as possible. There were no missing, corrupted or any illegible images found in the casting dataset that was why all the casting dataset were used for training purpose of proposed models.

7.2. Comparative Performance of CNN Models

Experimental results show that transfer learning with the MobileNetV2 model yields better performance compared to the Custom CNN on automated casting defect classification. In spite of achieving discriminant feature representations for casting defects with Custom CNN, it needs more epochs for training and obtains relative inferior performance in terms of classification. Instead, owing to the pretrained ImageNet feature representations, transfer learning approaches provide faster convergence rate and excellent performance in generalization ability and classification accuracy. The experimental results highlight that transfer learning offers an effective approach over the custom architecture CNN on binary casting defect classification.

7.3. Class-Level Reliability and Error Patterns

According to the confusion matrices and classification reports, both models obtained the same perfect (99.9%) precision and recall for both defect and non-defective casting products. Thus, both models were successfully applied to real-time visual defect inspection on industrial product images with almost all the casting products inspected accurately and reliably, proving the superiority of deep learning in such application. Nevertheless, there are still few misclassifications in either model.

These exceptions were mainly triggered by some images featuring surface defects with fine and indistinct texture, complex image backgrounds, and unstable lighting conditions, all of which could visually be mistaken for normal casting products.

Due to the nearly balanced nature of the training set, these errors seem to mainly arise from the complexity of image texture of these defect patterns rather than from a class imbalance.

7.4. Interpretation of Grad-CAM

Based on the result of Grad-CAM visualization, we can see that the designed networks tended to learn to focus on discriminative areas when generating their classification results. For defective casting samples, the resulted heatmaps put their emphasis on cracks, blowholes, porous surface, and other perceptible surface imperfections. On the other hand, for defect-free samples, activation maps emphasized on the surface of the casting and paid no significant attention to meaningless areas of the images. It builds up a higher

confidence of believing that the model decisions have been derived from authentic manufacturing defects, instead of irrelevant regions such as the background, making our casting defect detection method more interpretable and transparent.

7.5. Efficiency and Practical Deployment

The classification result was acceptable for both the two networks but mobileNetV2 have far less trainability parameters compared to Custom CNN and it converges really quickly. In addition, such well-organized architectural design also obtains higher rates of detection and lower inference latency because of the extremely lightness of such architecture. So the detection's higher accuracy, along with its faster inference, and lower computational cost make mobileNetV2 a preferable candidate for automatic quality control system and inspect in industry.

7.6. Comparison with Existing Literature

These experimental results are similar to recent experiments that highlight the benefits of transfer learning in automatic casting defect classification. Transfer learning has been proven to obtain considerable reduction of classification error as well as training time of CNNs, compared to traditional training approaches (e.g., custom training from scratch). In this article, Custom CNN and MobileNetV2 are also compared under the same training/testing set and protocol. Besides, the usage of Grad-CAM not only improves the interpretability of the models by indicating the location within an image that is responsible for a classification decision, but also provides further support and validation of our proposed inspection system.

7.6.1. Scientific Contributions

- Developed a comparative framework for evaluating a Custom CNN and MobileNetV2 for automated casting defect detection.
- Demonstrated the effectiveness of transfer learning for binary industrial defect classification.
- Integrated Grad-CAM to improve the interpretability and transparency of deep learning predictions.
- Established a reproducible deep learning workflow using TensorFlow/Keras within the Kaggle environment.

Practical Contributions

- Developed an automated casting defect detection system capable of distinguishing defective and non-defective casting products.
- Demonstrated that MobileNetV2 achieves an excellent balance between prediction accuracy and computational efficiency.
- Showed that Grad-CAM can provide meaningful visual explanations that assist industrial quality inspection.
- Presented an effective deep learning framework that can support manufacturing quality control and reduce manual inspection effort.

7.7. Ethical, Social, and Safety Implications

The dataset collected for this research comes from Kaggle platform containing only publicly available images of industrial casting parts. The dataset doesn't contain any personal information or privacy concerns of human subjects as the pictures only show the castings and don't reveal any sensitive or proprietary information of any entity. Even with good accuracy in classification, there may be few errors such as minor characteristics of the defect, change in light condition, quality of the image or manufacturing inconsistencies.

7.8. Limitations

While the developed models demonstrate impressive experimental results, it is worth considering the following limitations: First, the experiments were only performed on one available public casting defect image database, containing 7,348 images, which may not fully cover all kinds of casting product variations encountered in manufacturing settings.

Second, the public dataset covers variations in illumination, angle and texture in nature, yet does not fully consider difficult issues such as heavy noise, motion blur, partial occultations and substantial illumination changes commonly faced by real-world casting products.

Third, in this work, we only evaluated two popular CNN architectures: Custom CNN and MobileNetV2, without exploring more state-of-the-art deep learning models. Fourth, there was no external validation by industrial casting images, collected independently, hence, there is a need for the

designed casting defect identification system to undergo additional real-world production site test before applying to industrial practices.

Fifth, robustness of the designed casting defect detection models towards extremely challenging conditions and industrial variations has not been explored thoroughly.

7.9. Future Research Directions

Future research directions can involve exploring other transfer learning architectures such as EfficientNetV2, ConvNeXt, Vision Transformers (ViT) and Swin Transformers. An extended evaluation on larger and varied industrial casting datasets from other manufacturing plants would also give a thorough estimation of model generalisation performance. Moreover, object detection and instance segmentation models such as YOLOv11, Faster R-CNN, Mask R-CNN could also be examined to enable classifying defects and simultaneously providing their locations.

Also, further analysis on the robustness under severe industrial conditions such as changing light illumination, motion blur, image noise, dirt, partial occlusion, etc. Would reveal more on the trustworthiness of the models in a real industrial environment.

Other directions of future work include lightweight model compression techniques including pruning and quantisation for the models to run on embedded devices and edge computers for real time industrial quality control. Active learning and uncertainty estimation techniques could be incorporated to enhance the robustness and trustfulness of automated defect detection models.

8. Conclusion

This paper presented a deep learning and transfer learning based automated casting defect detection framework to classify industrial casting products into defective and non-defective classes for binary classification. The proposed framework includes dataset preprocessing, data augmentation, model construction, model training, performance evaluation and model interpretation by Grad-CAM. The paper presented experimental comparisons of a Custom Convolutional Neural Network (CNN) model and MobileNetV2 transfer learning based model. Results showed that MobileNetV2 outperformed the Custom CNN, reaching an overall test accuracy of 70% vs 80%. Besides high predictive performance, MobileNetV2 also had the advantages

of rapid convergence, low computational cost, and good generalization. Furthermore, Grad-CAM visualizations demonstrated that the models mainly attended to the defects of casting products and increased the interpretability of the models. In conclusion, this study explored an effective transfer learning framework and lightweight deep learning model for automatic detection of casting defects and shed lights on implementing deep learning into intelligent quality inspection of the real manufacturing system. Although we used one public dataset, this work provided a reproducible baseline for developing future methods in the context of much larger industrial datasets, sophisticated DL models, and validation with external sets and real industrial deployments.

Declarations

The authors used the generative AI tool ChatGPT (OpenAI, San Francisco, CA, USA) to improve the language and clarity of the manuscript. The authors reviewed and edited all content generated by the tool and take full responsibility for the final version of the manuscript.

References

- [1] S. Assad, N. A. M. Isa, S. A. M. Saleh, Hybrid cnn-transformer models for industrial defect detection: A systematic review, *Results in Engineering* 29 (2026) 109457. doi:10.1016/j.rineng.2026.109457. URL <https://www.sciencedirect.com/science/article/pii/S2590123026004974>
- [2] Y. Wang, M. Zhou, C. Zhang, K. Sedraoui, Applications of deep learning to metal surface defect detection: Status and challenges, *Processes* 14 (8) (2026). doi:10.3390/pr14081305. URL <https://www.mdpi.com/2227-9717/14/8/1305>
- [3] J. Plested, T. Gedeon, Deep transfer learning for image classification: A survey, *arXiv preprint arXiv:2205.09904* (2022).
- [4] M. Habibpour, H. Gharoun, A. Tajally, A. Shamsi, H. Asgharnezhad, A. Khosravi, S. Nahavandi, An uncertainty-aware deep learning framework for defect detection in casting products, *arXiv preprint arXiv:2107.11643* (2021).

- [5] H. Tanyıldız, E. B. Şahin, Transfer learning for detection of casting defects model in scope of industry 4.0, *Turkish Journal of Design for Engineering Fields* 9 (1) (2023) 35–44. doi:10.55071/tujef.1236584.
- [6] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, C. Liu, A survey on deep transfer learning, *arXiv preprint arXiv:1808.01974* (2018).
- [7] S. Assad, N. A. M. Isa, S. A. M. Saleh, Hybrid cnn-transformer models for industrial defect detection: A systematic review, *Results in Engineering* 29 (2026) 109457. doi:10.1016/j.rineng.2026.109457.
- [8] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mobilenetv2: Inverted residuals and linear bottlenecks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [9] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra, Grad-cam: Visual explanations from deep networks via gradient-based localization, in: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626. doi:10.1109/ICCV.2017.74.
- [10] R. Gupta, et al., Deep learning model for defect analysis in industry using ensemble transfer learning and convolutional neural networks, *Expert Systems with Applications* 228 (2023) 120376. doi:10.1016/j.eswa.2023.120376.

9. Implementation

Appendix A. Training Code

```
1  #!/usr/bin/env python
2  # coding: utf-8
3
4  # In[ ]:
5
6
7  # Casting Defect Detection using CNN
8
9  import numpy as np
10 import pandas as pd
11 import matplotlib.pyplot as plt
12 import seaborn as sns
13 import os
14
15 from tensorflow.keras.preprocessing.image import ImageDataGenerator, image
16 from tensorflow.keras.models import Sequential
17 from tensorflow.keras.layers import Conv2D, MaxPooling2D
18 from tensorflow.keras.layers import Flatten, Dense, Dropout
19 from tensorflow.keras.optimizers import Adam
20
21 from sklearn.metrics import classification_report
22 from sklearn.metrics import confusion_matrix
23 from sklearn.metrics import roc_curve, auc
24
25 import tensorflow as tf
26
27 print("TensorFlow Version:", tf.__version__)
28
29 # In[ ]:
30
31
32
33 train_dir = "casting_data/train"
34 test_dir = "casting_data/test"
35
36
37 # In[ ]:
38
39
40 print(os.listdir(train_dir))
41 print(os.listdir(test_dir))
42
43
44 # In[ ]:
45
46
47 from tensorflow.keras.preprocessing import image
48
49 img_path = os.path.join(
50     train_dir,
51     "def_front",
52     os.listdir(os.path.join(train_dir, "def_front"))[0]
```

```

53 )
54
55 img = image.load_img(img_path)
56
57 plt.imshow(img)
58 plt.axis("off")
59 plt.show()
60
61
62 # In[ ]:
63
64
65 train_datagen = ImageDataGenerator(
66     rescale=1./255,
67     validation_split=0.2,
68     rotation_range=20,
69     zoom_range=0.2,
70     horizontal_flip=True
71 )
72
73
74 # In[ ]:
75
76
77 train_generator = train_datagen.flow_from_directory(
78     train_dir,
79     target_size=(224,224),
80     batch_size=32,
81     class_mode='binary',
82     subset='training'
83 )
84
85
86 # In[ ]:
87
88
89 val_generator = train_datagen.flow_from_directory(
90     train_dir,
91     target_size=(224,224),
92     batch_size=32,
93     class_mode='binary',
94     subset='validation'
95 )
96
97
98 # In[ ]:
99
100
101 test_datagen = ImageDataGenerator(rescale=1./255)
102
103 test_generator = test_datagen.flow_from_directory(
104     test_dir,
105     target_size=(224,224),
106     batch_size=32,
107     class_mode='binary',
108     shuffle=False
109 )

```

```

110
111
112 # In[ ]:
113
114
115 model = Sequential()
116
117 model.add(
118     Conv2D(
119         32,
120         (3,3),
121         activation='relu',
122         input_shape=(224,224,3)
123     )
124 )
125
126 model.add(MaxPooling2D())
127
128 model.add(
129     Conv2D(
130         64,
131         (3,3),
132         activation='relu'
133     )
134 )
135
136 model.add(MaxPooling2D())
137
138 model.add(
139     Conv2D(
140         128,
141         (3,3),
142         activation='relu'
143     )
144 )
145
146 model.add(MaxPooling2D())
147
148 model.add(Flatten())
149
150 model.add(Dense(128, activation='relu'))
151
152 model.add(Dropout(0.5))
153
154 model.add(Dense(1, activation='sigmoid'))
155
156 model.summary()
157
158
159 # In[ ]:
160
161
162 model.compile(
163     optimizer='adam',
164     loss='binary_crossentropy',
165     metrics=['accuracy']
166 )

```

```

167
168
169 # In[ ]:
170
171
172 history = model.fit(
173     train_generator,
174     validation_data=val_generator,
175     epochs=15
176 )
177
178
179 # In[1]:
180
181
182 plt.plot(history.history['accuracy'])
183 plt.plot(history.history['val_accuracy'])
184
185 plt.title('Accuracy')
186 plt.xlabel('Epoch')
187 plt.ylabel('Accuracy')
188
189 plt.legend(['Train', 'Validation'])
190
191 plt.show()
192 plt.savefig("accuracy_curve.pdf")
193
194
195 # In[ ]:
196
197
198 plt.plot(history.history['loss'])
199 plt.plot(history.history['val_loss'])
200
201 plt.title('Loss')
202 plt.xlabel('Epoch')
203 plt.ylabel('Loss')
204
205 plt.legend(['Train', 'Validation'])
206
207 plt.show()
208 plt.savefig("loss_curve.png")
209
210
211 # In[ ]:
212
213
214 loss, accuracy = model.evaluate(test_generator)
215
216 print("Accuracy:", accuracy)
217
218
219 # In[ ]:
220
221
222 predictions = model.predict(test_generator)
223

```



```

224 y_pred = (predictions > 0.5).astype(int)
225
226
227 # In[ ]:
228
229
230 print(
231     classification_report(
232         test_generator.classes,
233         y_pred
234     )
235 )
236
237
238 # In[ ]:
239
240
241 cm = confusion_matrix(
242     test_generator.classes,
243     y_pred
244 )
245
246 sns.heatmap(
247     cm,
248     annot=True,
249     fmt='d',
250     cmap='Blues'
251 )
252
253 plt.title("Confusion Matrix")
254 plt.show()
255 plt.savefig("confusion_matrix.png")
256
257
258 # In[ ]:
259
260
261 from tensorflow.keras.applications import MobileNetV2
262
263
264 # In[ ]:
265
266
267 base_model = MobileNetV2(
268     weights='imagenet',
269     include_top=False,
270     input_shape=(224,224,3)
271 )
272 base_model.trainable = False
273
274
275 # In[ ]:
276
277
278 from tensorflow.keras.layers import GlobalAveragePooling2D
279
280 transfer_model = Sequential([

```

```

281         base_model,
282         GlobalAveragePooling2D(),
283         Dense(128, activation='relu'),
284         Dropout(0.5),
285         Dense(1, activation='sigmoid')
286     ])
287
288
289 # In[ ]:
290
291
292 transfer_model.compile(
293     optimizer='adam',
294     loss='binary_crossentropy',
295     metrics=['accuracy']
296 )
297
298 transfer_history = transfer_model.fit(
299     train_generator,
300     validation_data=val_generator,
301     epochs=10
302 )
303
304
305 # In[ ]:
306
307
308 loss, cnn_acc = model.evaluate(test_generator)
309 print("CNN Accuracy:", cnn_acc)
310
311 mobile_loss, mobile_acc = transfer_model.evaluate(test_generator)
312 print("MobileNetV2 Accuracy:", mobile_acc)
313
314
315 # In[ ]:
316
317
318 model.save("cnn_model.h5")
319
320 transfer_model.save("mobilenet_model.h5")
321
322
323 # In[ ]:
324
325
326 transfer_model.summary()
327
328
329 # In[ ]:
330
331
332 img_path = os.path.join(
333     test_dir,
334     "def_front",
335     os.listdir(os.path.join(test_dir, "def_front"))[0]
336 )
337

```

```

338 img = image.load_img(img_path, target_size=(224,224))
339
340 plt.imshow(img)
341 plt.axis("off")
342 plt.show()
343
344
345 # In[ ]:
346
347
348 img_array = image.img_to_array(img)
349 img_array = np.expand_dims(img_array, axis=0)
350 img_array = img_array / 255.0
351
352 prediction = transfer_model.predict(img_array)
353
354 print(prediction)
355
356
357 # In[ ]:
358
359
360 base_model = transfer_model.layers[0]
361
362 feature_model = tf.keras.Model(
363     inputs=base_model.input,
364     outputs=base_model.get_layer("Conv_1").output
365 )
366
367 feature_maps = feature_model.predict(img_array)
368
369 print(feature_maps.shape)
370 print(feature_maps.min())
371 print(feature_maps.max())
372
373
374 # In[ ]:
375
376
377 inputs = transfer_model.input
378
379 x = transfer_model.layers[0](inputs)
380 conv_output = x
381
382 x = transfer_model.layers[1](x)
383 x = transfer_model.layers[2](x)
384 x = transfer_model.layers[3](x)
385 predictions = transfer_model.layers[4](x)
386
387 grad_model = tf.keras.models.Model(
388     inputs=inputs,
389     outputs=[conv_output, predictions]
390 )
391
392 print("Grad model created successfully")
393
394

```

```

395 # In[ ]:
396
397
398
399
400
401 # In[ ]:
402
403
404 for layer in model.layers:
405     print(layer.name)
406 print("Mean:", heatmap.mean())
407
408
409 # In[ ]:
410
411
412 conv_outputs, preds = grad_model(img_array)
413
414 print(conv_outputs.shape)
415 print(preds.numpy())
416
417
418 # In[ ]:
419
420
421 with tf.GradientTape() as tape:
422
423     conv_outputs, predictions = grad_model(img_array)
424
425     loss = predictions[:, 0]
426
427 grads = tape.gradient(
428     loss,
429     conv_outputs
430 )
431
432 print("Gradient shape:", grads.shape)
433 print("Gradient max:", tf.reduce_max(grads).numpy())
434 print("Gradient min:", tf.reduce_min(grads).numpy())
435
436
437 # In[ ]:
438
439
440 pooled_grads = tf.reduce_mean(
441     grads,
442     axis=(0,1,2)
443 )
444
445 conv_outputs = conv_outputs[0]
446
447 heatmap = tf.reduce_sum(
448     pooled_grads * conv_outputs,
449     axis=-1
450 )
451

```

```

452 heatmap = tf.maximum(
453     heatmap,
454     0
455 )
456
457 heatmap = heatmap / (
458     tf.reduce_max(heatmap) + 1e-10
459 )
460
461 heatmap = heatmap.numpy()
462
463 print("Shape:", heatmap.shape)
464 print("Min:", heatmap.min())
465 print("Max:", heatmap.max())
466 print("Mean:", heatmap.mean())
467
468 # In[ ]:
469
470
471 plt.figure(figsize=(6,6))
472 plt.imshow(
473     heatmap,
474     cmap="jet"
475 )
476 plt.colorbar()
477 plt.title("Grad-CAM Heatmap")
478 plt.show()
479
480
481 # In[ ]:
482
483
484 from PIL import Image
485
486 original_img = Image.open(img_path)
487
488 original_img = np.array(original_img)
489
490 heatmap_resized = np.array(
491     Image.fromarray(
492         np.uint8(heatmap * 255)
493     ).resize(
494         (original_img.shape[1], original_img.shape[0])
495     )
496 )
497
498 plt.figure(figsize=(8,8))
499 plt.imshow(heatmap_resized, cmap='jet')
500 plt.colorbar()
501 plt.title("Grad-CAM Heatmap")
502 plt.show()
503
504
505 # In[ ]:
506
507
508

```

```

509 original_img = np.array(Image.open(img_path))
510
511 heatmap_resized = np.array(
512     Image.fromarray(
513         np.uint8(heatmap * 255)
514     ).resize(
515         (original_img.shape[1],
516          original_img.shape[0])
517     )
518 )
519
520 plt.figure(figsize=(10,10))
521
522 plt.imshow(original_img)
523
524 plt.imshow(
525     heatmap_resized,
526     cmap='jet',
527     alpha=0.4
528 )
529
530 plt.axis("off")
531 plt.title("MobileNetV2 Grad-CAM Overlay")
532
533 plt.show()
534
535
536 # In[ ]:
537
538
539 plt.figure(figsize=(10,10))
540
541 plt.imshow(original_img)
542
543 plt.imshow(
544     heatmap_resized,
545     cmap='jet',
546     alpha=0.4
547 )
548
549 plt.axis("off")
550
551 plt.savefig(
552     "gradcam_mobilenetv2_overlay.png",
553     dpi=300,
554     bbox_inches="tight"
555 )
556
557 plt.show()

```

Listing 1: Training implementation for the proposed Custom CNN and MobileNetV2 models

Appendix B. Testing Code

```
1 #!/usr/bin/env python
2  # coding: utf-8
3
4  # In[1]:
5
6
7  import os
8  import numpy as np
9  import matplotlib.pyplot as plt
10
11 from tensorflow.keras.models import load_model
12 from tensorflow.keras.preprocessing.image import ImageDataGenerator
13
14 from sklearn.metrics import (
15     accuracy_score,
16     precision_score,
17     recall_score,
18     f1_score,
19     classification_report,
20     confusion_matrix,
21     ConfusionMatrixDisplay
22 )
23
24
25 # In[2]:
26
27
28 TEST_PATH = r"E:\Casting defect classification framework\Dataset\CNN test"
29
30 IMG_SIZE = (224,224)
31 BATCH_SIZE = 32
32
33
34 # In[3]:
35
36
37 test_datagen = ImageDataGenerator(rescale=1./255)
38
39 test_generator = test_datagen.flow_from_directory(
40     TEST_PATH,
41     target_size=IMG_SIZE,
42     batch_size=BATCH_SIZE,
43     class_mode='binary',
44     shuffle=False
45 )
46
47 print("Classes:", test_generator.class_indices)
48
49
50 # In[4]:
51
52
53 cnn_model = load_model("E:\Casting defect classification framework\cnn_model
54     .h5")
```

```

55 mobilenet_model = load_model("E:\Casting defect classification framework\
    mobilenet_model.h5")
56
57 print("Both models loaded successfully.")
58
59
60 # In[5]:
61
62
63 cnn_loss, cnn_accuracy = cnn_model.evaluate(test_generator)
64
65 print("\nCustom CNN Results")
66 print("-----")
67 print(f"Loss      : {cnn_loss:.4f}")
68 print(f"Accuracy : {cnn_accuracy*100:.2f}%")
69
70
71 # In[6]:
72
73
74 test_generator.reset()
75
76 cnn_predictions = cnn_model.predict(test_generator)
77
78 cnn_predicted = (cnn_predictions > 0.5).astype(int).flatten()
79
80 true_classes = test_generator.classes
81
82
83 # In[7]:
84
85
86 print("CUSTOM CNN PERFORMANCE")
87 print("="*50)
88
89 print("Accuracy :", accuracy_score(true_classes, cnn_predicted))
90 print("Precision:", precision_score(true_classes, cnn_predicted))
91 print("Recall   :", recall_score(true_classes, cnn_predicted))
92 print("F1 Score :", f1_score(true_classes, cnn_predicted))
93
94
95 # In[8]:
96
97
98 print(classification_report(
99     true_classes,
100     cnn_predicted,
101     target_names=['Defective', 'OK']
102 ))
103
104
105 # In[9]:
106
107
108 cm = confusion_matrix(true_classes, cnn_predicted)
109
110 disp = ConfusionMatrixDisplay(

```



```

111     confusion_matrix=cm,
112     display_labels=['Defective','OK']
113 )
114
115 disp.plot(cmap="Blues")
116
117 plt.title("Custom CNN Confusion Matrix")
118 plt.show()
119
120
121 # In[10]:
122
123
124 mobilenet_loss,mobilenet_accuracy = mobilenet_model.evaluate(test_generator)
125
126 print("\nMobileNetV2 Results")
127 print("-----")
128 print(f"Loss      : {mobilenet_loss:.4f}")
129 print(f"Accuracy : {mobilenet_accuracy*100:.2f}%")
130
131
132 # In[11]:
133
134
135 test_generator.reset()
136
137 mobile_predictions = mobilenet_model.predict(test_generator)
138
139 mobile_predicted = (mobile_predictions>0.5).astype(int).flatten()
140
141
142 # In[12]:
143
144
145 print("MOBILENETV2 PERFORMANCE")
146 print("="*50)
147
148 print("Accuracy :",accuracy_score(true_classes,mobile_predicted))
149 print("Precision:",precision_score(true_classes,mobile_predicted))
150 print("Recall    :",recall_score(true_classes,mobile_predicted))
151 print("F1 Score  :",f1_score(true_classes,mobile_predicted))
152
153
154 # In[13]:
155
156
157 print(classification_report(
158     true_classes,
159     mobile_predicted,
160     target_names=['Defective','OK']
161 ))
162
163
164 # In[14]:
165
166
167 cm = confusion_matrix(true_classes,mobile_predicted)

```

```

168
169 disp = ConfusionMatrixDisplay(
170     confusion_matrix=cm,
171     display_labels=['Defective','OK']
172 )
173
174 disp.plot(cmap="Greens")
175
176 plt.title("MobileNetV2 Confusion Matrix")
177 plt.show()
178
179
180 # In[15]:
181
182
183 print("="*60)
184 print("MODEL COMPARISON")
185 print("="*60)
186
187 print(f"Custom CNN Accuracy      : {cnn_accuracy*100:.2f}%")
188 print(f"MobileNetV2 Accuracy      : {mobilenet_accuracy*100:.2f}%")
189
190 print(f"Custom CNN Loss            : {cnn_loss:.4f}")
191 print(f"MobileNetV2 Loss            : {mobilenet_loss:.4f}")
192
193
194 # In[16]:
195
196
197 #generate loss curve and accuracy curve for both models
198
199
200 # In[17]:
201
202
203 class_names = ['Defective','OK']
204
205 for class_name in class_names:
206
207     folder = os.path.join(TEST_PATH,class_name)
208
209     for filename in os.listdir(folder):
210
211         image_path = os.path.join(folder,filename)
212
213         from tensorflow.keras.preprocessing import image
214
215         img = image.load_img(image_path,target_size=(224,224))
216
217         x = image.img_to_array(img)/255.0
218         x = np.expand_dims(x,axis=0)
219
220         cnn_pred = cnn_model.predict(x,verbose=0)
221         mobile_pred = mobilenet_model.predict(x,verbose=0)
222
223         cnn_class = class_names[int(cnn_pred[0][0]>0.5)]
224         mobile_class = class_names[int(mobile_pred[0][0]>0.5)]

```

```

225
226     cnn_conf = cnn_pred[0][0] if cnn_class=="OK" else 1-cnn_pred[0][0]
227     mobile_conf = mobile_pred[0][0] if mobile_class=="OK" else 1-
        mobile_pred[0][0]
228
229     plt.figure(figsize=(5,5))
230     plt.imshow(img)
231     plt.axis("off")
232
233     plt.title(
234         f"Actual : {class_name}\n\n"
235         f"CNN : {cnn_class} ({cnn_conf*100:.2f}%) \n"
236         f"MobileNetV2 : {mobile_class} ({mobile_conf*100:.2f}%) "
237     )
238
239     plt.show()
240
241
242 # In[18]:
243
244
245 TEST_PATH = r"E:\Casting defect classification framework\Dataset\CNN test"
246 IMG_SIZE = (224, 224)
247 class_names = ["Defective", "OK"]
248
249 model = cnn_model
250
251 images = []
252
253 for actual_class in class_names:
254
255     folder = os.path.join(TEST_PATH, actual_class)
256
257     for filename in sorted(os.listdir(folder)):
258
259         img_path = os.path.join(folder, filename)
260
261         img = image.load_img(img_path, target_size=IMG_SIZE)
262
263         x = image.img_to_array(img)
264         x = x / 255.0
265         x = np.expand_dims(x, axis=0)
266
267         prediction = model.predict(x, verbose=0)
268
269         probability = prediction[0][0]
270
271         if probability >= 0.5:
272             predicted = "OK"
273             confidence = probability * 100
274         else:
275             predicted = "Defective"
276             confidence = (1 - probability) * 100
277
278         status = "Correct" if predicted == actual_class else "Wrong"
279
280         images.append({

```

```

281         "image": img,
282         "actual": actual_class,
283         "predicted": predicted,
284         "confidence": confidence,
285         "status": status
286     })
287
288     cols = 5
289     rows = math.ceil(len(images) / cols)
290
291     fig, axes = plt.subplots(rows, cols, figsize=(18, rows * 4))
292     axes = axes.flatten()
293
294     for i, item in enumerate(images):
295
296         axes[i].imshow(item["image"])
297         axes[i].axis("off")
298
299         title_color = "green" if item["status"] == "Correct" else "red"
300
301         axes[i].set_title(
302             f"Actual : {item['actual']}\n"
303             f"Predicted : {item['predicted']}\n"
304             f"Confidence : {item['confidence']:.2f}%\n"
305             f"Status : {item['status']}",
306             fontsize=8,
307             color=title_color
308         )
309
310     # Hide empty plots
311     for j in range(len(images), len(axes)):
312         axes[j].axis("off")
313
314     plt.suptitle(
315         "Casting Defect Detection - Prediction Results\n",
316         fontsize=20,
317         fontweight="bold"
318     )
319
320     plt.tight_layout()
321
322     # Save as PDF
323     plt.savefig(
324         r"E:\Casting defect classification framework\Casting_Defect_Predictions.
325         pdf",
326         format="pdf",
327         dpi=300,
328         bbox_inches="tight"
329     )
330
331     plt.show()
332     print("\nPDF saved successfully!")

```

Listing 2: Testing and evaluation implementation